Personalised IT Career Planner

With Skill-Gap Insights and Course Recommendations

Field	Detail
Student	Hein Htet Phyo
Student ID	25036746
Module	UFCEKP-30-3 — Data Science and AI Individual Project
GitHub	HeinHtet-Phyo / it-career-planner

Problem Statement

Many students and self-learners who want to work in IT do not know which specific role genuinely matches their interests and current abilities. Generic career advice rarely explains what exact skills each role requires or how to build them systematically. At the same time, the volume of available online learning resources is overwhelming, and learners do not know where to start.

This project addresses that problem by building a **Personalised IT Career Planner** — a data-driven, end-to-end system that takes a structured learner profile and produces personalised, actionable career guidance through a supervised machine learning pipeline combined with rule-based analytical output.

System Pipeline

Stage	Component	Description
1	Profile Input	Learner provides skill scores (1–5), interest levels, work preference, learning style,
2	ML Classification	Decision Tree / Random Forest / XGBoost predicts best-fit roles
3	Top 3 Roles	Top 3 recommended roles with readiness scores
4	Role Selection	Learner selects one role to explore
5	Skill-Gap Analysis	Current vs required skills — gaps classified and prioritised
6	Why-It-Fits	Rule-based personalised explanation of why the role matches
7	Roadmap	Ordered learning plan based on gap priority
8	Resources	Curated resources matched by skill, level, and role

Five Datasets

Dataset	Rows	Role in System	Justificat
`user_profiles.csv`	6,000	Main ML training dataset	Provides labelled learner profiles for su
`role_skill_matrix.csv`	6	Readiness and gap scoring	Defines authoritative skill requirements
`role_metadata.csv`	6	Role output explanations	Provides human-readable role descrip
`roadmap_templates.csv`	36	Roadmap generation	Contains ordered learning step templa
`resource_catalogue.csv`	300	Resource recommendations	Curated resources matched by skill, lev

> **Dataset Design Justification:** All five datasets were self-constructed because no single public dataset covers the full workflow of learner-profile role prediction, skill-gap analysis, roadmap generation, and resource mapping. Each dataset serves a clearly defined and distinct purpose in the pipeline, avoiding redundancy.

Section 1: Setup and Data Loading

All libraries, colour palettes, and display settings are configured here to ensure a consistent, professional visual identity across the entire notebook. Centralising setup avoids repetition and makes the notebook easier to maintain.

```
!pip install xgboost --quiet
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remo
```

```
import os
import re
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import FancyBboxPatch
import seaborn as sns
import warnings
```

```
warnings.filterwarnings('ignore')

# Machine learning — training, evaluation, preprocessing
from sklearn.model_selection import train_test_split, StratifiedKFc
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    f1_score, precision_score, recall_score
)
from xgboost import XGBClassifier

# Colab interactive widgets for the UI demo
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output

# ── Global display and plot settings ──────────────────────────────
pd.set_option('display.max_columns', None)
pd.set_option('display.max_colwidth', 90)
plt.rcParams.update({
    'figure.dpi': 120,
    'font.family': 'DejaVu Sans',
    'axes.spines.top': False,
    'axes.spines.right': False
})

# ── Project colour palette — one distinct colour per IT role ─────
# Consistent across every chart in this notebook
ROLE_COLORS  = ['#2563EB','#7C3AED','#DB2777','#EA580C','#16A34A','
SKILL_COLORS = ['#EF4444','#F97316','#EAB308','#22C55E',
                '#06B6D4','#3B82F6','#8B5CF6','#EC4899','#14B8A6','
GAP_CURRENT  = '#3B82F6'
GAP_REQUIRED = '#EC4899'

print('Setup complete.')
```

```
Setup complete.
```

```
# Load all five datasets
# Auto-detects: Google Colab (Drive) or GitHub/local (data/ folder)

if os.path.exists('/content/drive/MyDrive/IT_Career_Planner_Project
    BASE = '/content/drive/MyDrive/IT_Career_Planner_Project/'
    print("✅ Running in Google Colab — loading from Google Drive")
else:
    BASE = 'data/'
    print("✅ Running locally/GitHub — loading from data/ folder")

user_profiles      = pd.read_csv(BASE + 'user_profiles.csv')
role_skill_matrix  = pd.read_csv(BASE + 'role_skill_matrix.csv')
role_metadata      = pd.read_csv(BASE + 'role_metadata.csv')
roadmap_templates  = pd.read_csv(BASE + 'roadmap_templates.csv')
resource_catalogue = pd.read_csv(BASE + 'resource_catalogue.csv')

print('All datasets loaded successfully.')
for name, df in [('user_profiles',user_profiles),('role_skill_matri
                 ('role_metadata',role_metadata),('roadmap_template
                 ('resource_catalogue',resource_catalogue)]:
    print(f'  {name:22s}: {df.shape[0]:>5} rows x {df.shape[1]} col
```

```
✅ Running in Google Colab — loading from Google Drive
All datasets loaded successfully.
  user_profiles         :  6000 rows x 21 cols
  role_skill_matrix     :     6 rows x 16 cols
  role_metadata         :     6 rows x 6 cols
  roadmap_templates     :    36 rows x 5 cols
  resource_catalogue    :   300 rows x 8 cols
```

Section 2: Dataset Overview

Before any cleaning or modelling, we inspect the structure of the main dataset to understand the feature design, confirm data types, and identify which columns serve which role in the pipeline.

The `user_profiles.csv` dataset is designed with five feature groups:

- **Interest scores** (6 columns) — self-rated interest in each domain area (1–5)
- **Skill scores** (10 columns) — self-rated technical skill level per skill (1–5)
- **Preferred work type** — the type of work environment preferred
- **Learning style** — preferred mode of learning
- **Career goal** — the learner's stated career objective
- **Target role** — the supervised learning label (ground truth)

```
print(f'Shape: {user_profiles.shape}')
user_profiles.head(3)
```

```
Shape: (6000, 21)
```

	user_id	interest_data_analysis	interest_ai_ml	interest_software
0	U00001	4	4	
1	U00002	4	4	
2	U00003	5	2	

```
# Define feature groups — used consistently throughout the notebook
skill_cols    = [c for c in user_profiles.columns if c.startswith('
interest_cols = [c for c in user_profiles.columns if c.startswith('
cat_cols      = ['preferred_work_type','preferred_learning_style','
skill_labels  = [c.replace('skill_','').replace('_',' ').title() fo
interest_labels = [c.replace('interest_','').replace('_',' ').title

print(f'Interest features  ({len(interest_cols)}): {interest_cols}'
print(f'Skill features     ({len(skill_cols)}):    {skill_cols}')
print(f'Categorical        ({len(cat_cols)}):      {cat_cols}')
print(f'Target column: target_role')
print(f'\nData types:')
print(user_profiles.dtypes)
```

```
Interest features  (6): ['interest_data_analysis', 'interest_ai_ml',
Skill features     (10):    ['skill_python', 'skill_sql', 'skill_sta
Categorical        (3):      ['preferred_work_type', 'preferred_lear
Target column: target_role

Data types:
user_id                             object
interest_data_analysis               int64
interest_ai_ml                       int64
interest_software_development        int64
interest_web_development             int64
interest_cloud_infrastructure        int64
interest_business_problem_solving    int64
skill_python                         int64
skill_sql                            int64
skill_statistics                     int64
skill_machine_learning               int64
skill_programming_fundamentals       int64
skill_web_development                int64
skill_cloud                          int64
skill_devops                         int64
skill_data_visualisation             int64
skill_git                            int64
preferred_work_type                 object
preferred_learning_style            object
career_goal                         object
target_role                         object
dtype: object
```

Section 3: Data Cleaning and Validation

Data quality validation is a critical prerequisite to reliable modelling. Poor-quality data can introduce bias, reduce model accuracy, and produce misleading results. We systematically check for:

- **Missing values** — nulls would break model training or introduce bias
- **Duplicate rows** — exact duplicates inflate training frequency and distort evaluation metrics
- **Out-of-range scores** — skill and interest scores must be within 1–5 by design
- **Class balance** — imbalanced classes require different evaluation strategies (e.g. weighted F1)
- **Categorical validity** — confirm all categorical values match expected options

A clean, validated dataset gives confidence that the ML models are learning genuine patterns from the data rather than artefacts of data quality issues.

```
# Missing value check across all columns
missing = user_profiles.isnull().sum()
print('Missing values:',
      'None found.' if missing.sum()==0 else missing[missing>0].to_
```

```
Missing values: None found.
```

```
# Duplicate row check — remove if found to prevent training bias
dupes = user_profiles.duplicated().sum()
if dupes > 0:
    user_profiles = user_profiles.drop_duplicates().reset_index(drc
    print(f'{dupes} duplicate(s) removed. New shape: {user_profiles
else:
    print('No duplicate rows found.')
```

```
No duplicate rows found.
```

```
# Out-of-range validation — all skill and interest scores must be 1
numeric_cols = skill_cols + interest_cols
issues = {c: int(((user_profiles[c]<1)|(user_profiles[c]>5)).sum())
          for c in numeric_cols
          if ((user_profiles[c]<1)|(user_profiles[c]>5)).sum() > 0}
print('Out-of-range issues:', 'None.' if not issues else issues)
```

```
Out-of-range issues: None.
```

```
# Summary statistics — verify score distributions are reasonable
print('Summary statistics for all numeric features:')
user_profiles[numeric_cols].describe().round(2)
```

```
Summary statistics for all numeric features:
```

	skill_python	skill_sql	skill_statistics	skill_machine_learn
count	6000.00	6000.00	6000.00	600
mean	3.52	2.71	2.65	
std	1.13	1.36	1.32	
min	1.00	1.00	1.00	
25%	3.00	2.00	1.00	
50%	4.00	3.00	3.00	
75%	4.00	4.00	4.00	
max	5.00	5.00	5.00	

```
# Class balance check — balanced classes do not require weighted sa
role_counts = user_profiles['target_role'].value_counts()
print('Target role class distribution:')
print(role_counts.to_string())
print(f'\nDataset is perfectly balanced: {role_counts.iloc[0]} prof
print('Implication: Macro F1-score is appropriate as the primary ev
```

```
Target role class distribution:
target_role
Data Analyst                 1000
Data Scientist               1000
Machine Learning Engineer    1000
Software Engineer            1000
Web Developer                1000
Cloud/DevOps Engineer        1000

Dataset is perfectly balanced: 1000 profiles per role, 6,000 total.
Implication: Macro F1-score is appropriate as the primary evaluatior
```

```
# Validate categorical column values match expected options
for col in cat_cols:
    print(f'{col}: {user_profiles[col].unique().tolist()}')
print('\nAll categorical values are valid and consistent.')
```

```
preferred_work_type: ['Analytical', 'Research-Oriented', 'System Bui
preferred_learning_style: ['Visual', 'Practical', 'Project-Based', '
career_goal: ['Exploring Options', 'Work with Data', 'Build AI Syste

All categorical values are valid and consistent.
```

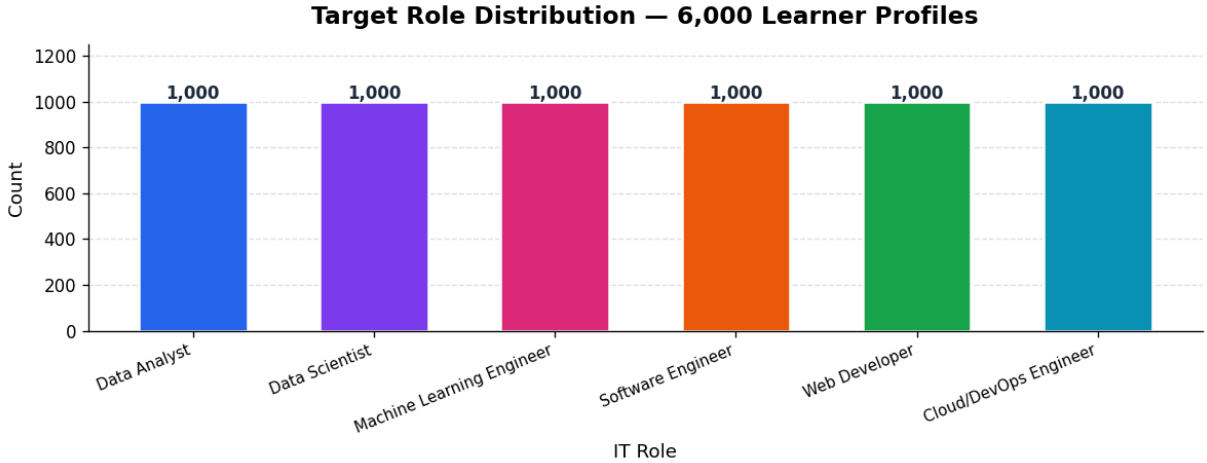
Section 4: Exploratory Data Analysis

EDA allows us to understand the structure, distributions, and relationships in the data before any modelling decisions are made. The key questions we address are:

1. Is the dataset balanced across roles? *(determines which evaluation metric is appropriate)*
2. Does each role have a distinct skill signature? *(confirms skill features carry discriminatory power)*
3. Does each role have a distinct interest signature? *(confirms interest features add additional signal)*
4. Are skill and interest features correlated? *(informs feature selection)*
5. Do qualitative features align with target roles? *(justifies including them as features)*

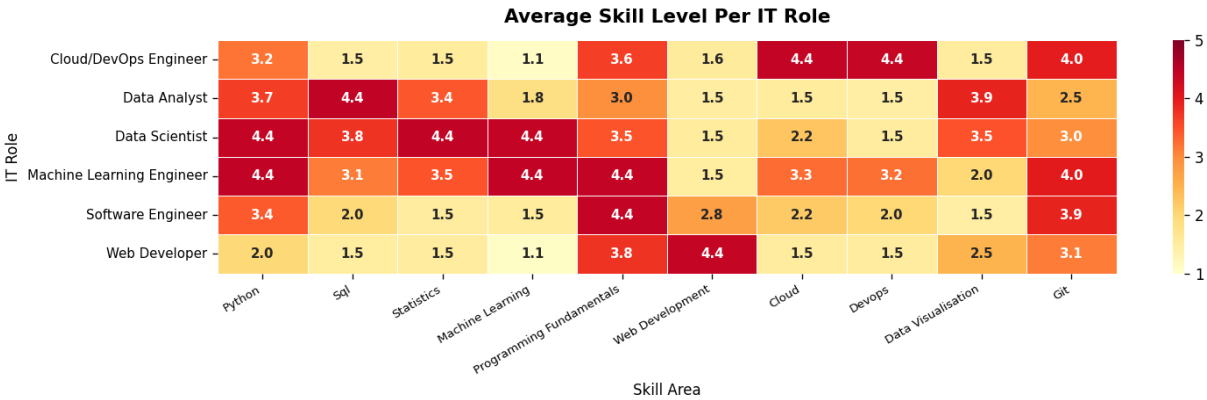
Each chart below is followed by an interpretation that directly informs the modelling approach.

```
# ── 4.1 Class distribution — confirms balanced dataset ──────────
fig, ax = plt.subplots(figsize=(10, 4))
bars = ax.bar(role_counts.index, role_counts.values,
              color=ROLE_COLORS, edgecolor='white', linewidth=1.5,
for bar in bars:
    ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+12,
            f'{int(bar.get_height()):,}', ha='center',
            fontsize=10, fontweight='bold', color='#1E293B')
ax.set_title('Target Role Distribution — 6,000 Learner Profiles',
             fontsize=14, fontweight='bold', pad=12)
ax.set_xlabel('IT Role', fontsize=11)
ax.set_ylabel('Count', fontsize=11)
ax.set_xticklabels(role_counts.index, rotation=22, ha='right', font
ax.set_ylim(0, 1250)
ax.yaxis.grid(True, linestyle='--', alpha=0.4)
ax.set_axisbelow(True)
plt.tight_layout()
plt.show()
print('Interpretation: The dataset is perfectly balanced with exact
      ' This confirms that no class imbalance correction is needed
      ' is the appropriate evaluation metric, giving equal weight t
```



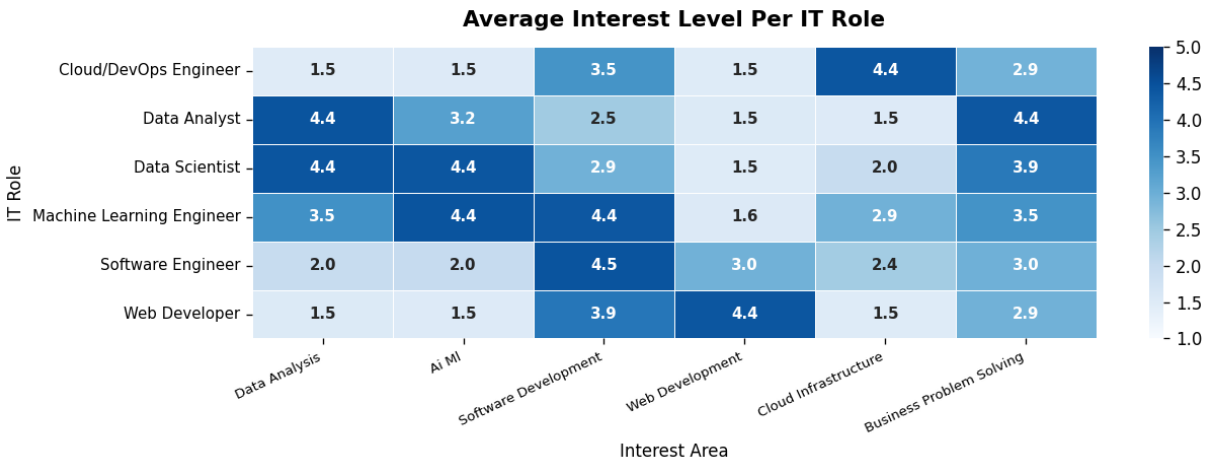

```
Interpretation: The dataset is perfectly balanced with exactly 1,000
```

```
# ── 4.2 Average skill profile per role — heatmap ──────────────────
# This is the most important EDA chart — shows each role has a dist
skill_by_role = user_profiles.groupby('target_role')[skill_cols].me
skill_by_role.columns = skill_labels

fig, ax = plt.subplots(figsize=(13, 4))
sns.heatmap(skill_by_role, annot=True, fmt='.1f',
            cmap='YlOrRd', vmin=1, vmax=5,
            linewidths=0.5, linecolor='white',
            annot_kws={'size':9,'weight':'bold'}, ax=ax)
ax.set_title('Average Skill Level Per IT Role',
             fontsize=13, fontweight='bold', pad=12)
ax.set_xlabel('Skill Area', fontsize=10)
ax.set_ylabel('IT Role', fontsize=10)
plt.xticks(rotation=30, ha='right', fontsize=8)
plt.yticks(rotation=0, fontsize=9)
plt.tight_layout()
plt.show()
print('Interpretation: Each role shows a clearly distinct skill sig
      ' Data Analysts score highest in SQL and Data Visualisation,
      ' score highest in Cloud and DevOps. This confirms that skill
      ' discriminatory power for multi-class classification.')
```




```
Interpretation: Each role shows a clearly distinct skill signature.
```

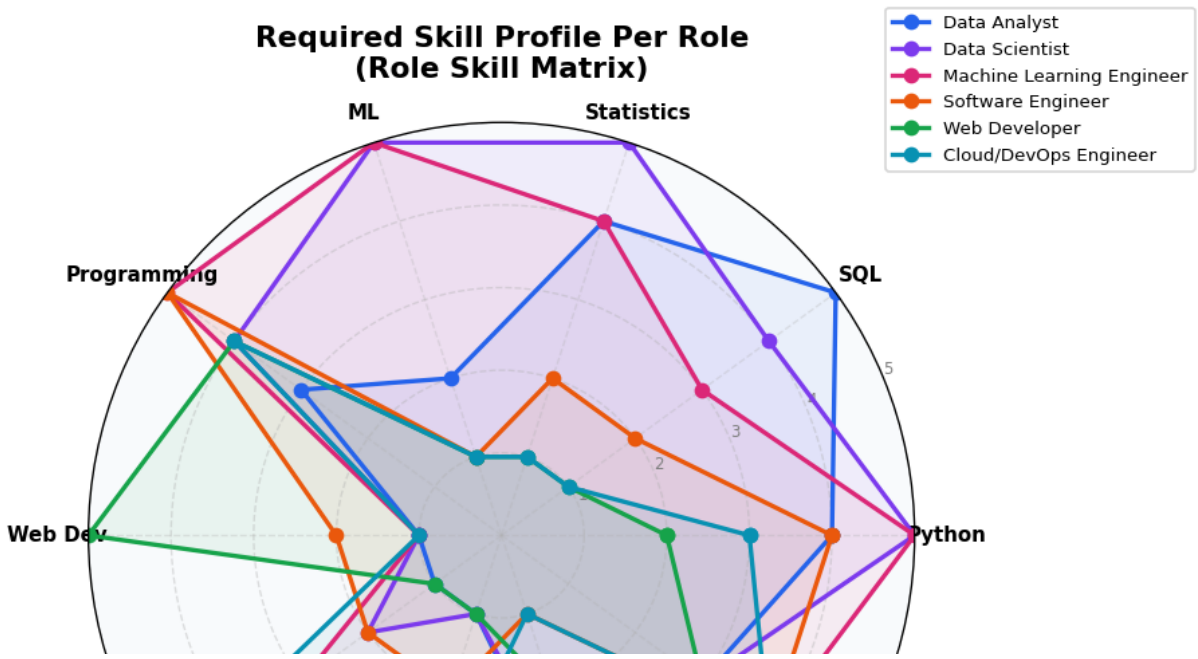
```
# ── 4.3 Average interest profile per role — heatmap ──────────────
# Justifies including interest features alongside skill features
int_by_role = user_profiles.groupby('target_role')[interest_cols].m
int_by_role.columns = interest_labels

fig, ax = plt.subplots(figsize=(11, 4))
sns.heatmap(int_by_role, annot=True, fmt='.1f',
            cmap='Blues', vmin=1, vmax=5,
            linewidths=0.5, linecolor='white',
            annot_kws={'size':9,'weight':'bold'}, ax=ax)
ax.set_title('Average Interest Level Per IT Role',
             fontsize=13, fontweight='bold', pad=12)
ax.set_xlabel('Interest Area', fontsize=10)
ax.set_ylabel('IT Role', fontsize=10)
plt.xticks(rotation=25, ha='right', fontsize=8)
plt.yticks(rotation=0, fontsize=9)
plt.tight_layout()
plt.show()
print('Interpretation: Interest profiles are also role-specific. Da
      ' high AI/ML interest while Web Developers show high web deve
      ' This justifies including interest features as predictors al
```

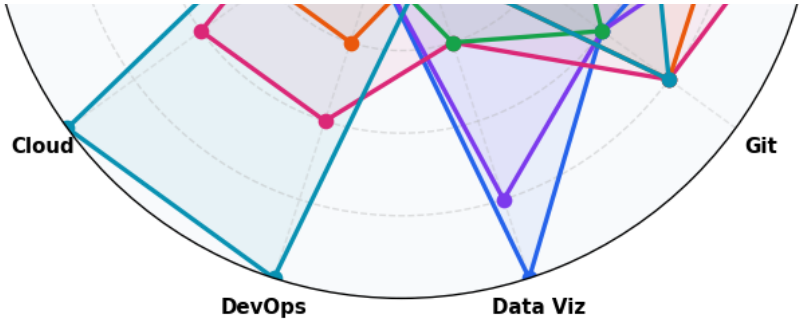



```
Interpretation: Interest profiles are also role-specific. Data Scien
```

```
# ── 4.4 Radar chart — role skill requirement comparison ──────────
# Spider chart showing the required skill profile for each role fro
req_cols  = ['python_required','sql_required','statistics_required'
             'programming_required','web_required','cloud_required'
             'devops_required','data_viz_required','git_required']
req_labels = ['Python','SQL','Statistics','ML','Programming',
```

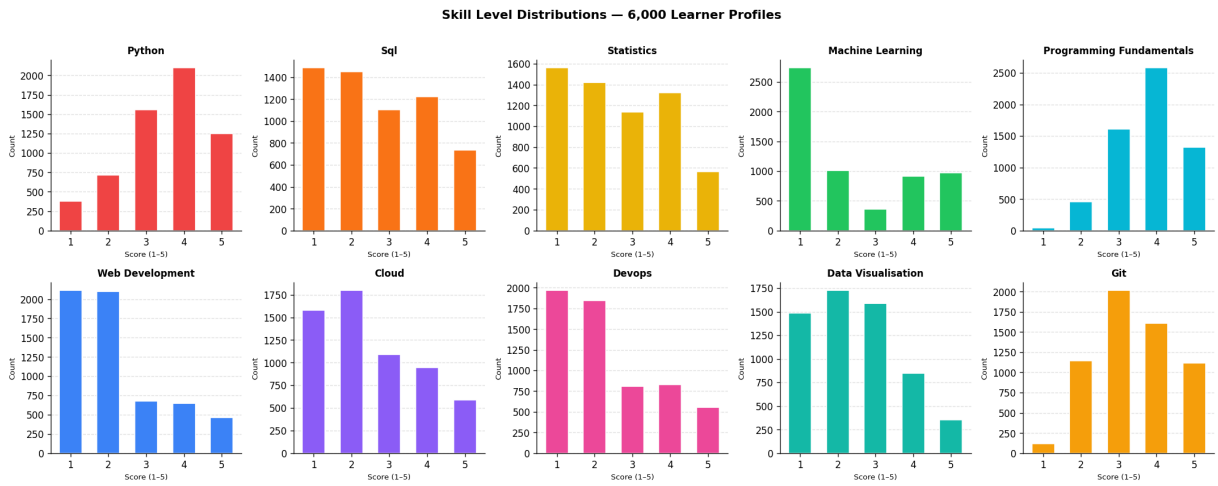
```
                 'Web Dev','Cloud','DevOps','Data Viz','Git']

N      = len(req_labels)
angles = np.linspace(0, 2*np.pi, N, endpoint=False).tolist()
angles += angles[:1]  # close the polygon

fig, ax = plt.subplots(figsize=(8, 8), subplot_kw=dict(polar=True))
ax.set_facecolor('#F8FAFC')

for i, (_, row) in enumerate(role_skill_matrix.iterrows()):
    values = [row[c] for c in req_cols]
    values += values[:1]
    ax.plot(angles, values, 'o-', linewidth=2,
            color=ROLE_COLORS[i], label=row['role'])
    ax.fill(angles, values, alpha=0.07, color=ROLE_COLORS[i])

ax.set_xticks(angles[:-1])
ax.set_xticklabels(req_labels, fontsize=9, fontweight='bold')
ax.set_ylim(0, 5)
ax.set_yticks([1,2,3,4,5])
ax.set_yticklabels(['1','2','3','4','5'], fontsize=7, color='grey')
ax.yaxis.grid(True, linestyle='--', alpha=0.4)
ax.xaxis.grid(True, linestyle='--', alpha=0.3)
ax.set_title('Required Skill Profile Per Role\n(Role Skill Matrix)'
             fontsize=13, fontweight='bold', pad=20)
ax.legend(loc='upper right', bbox_to_anchor=(1.35, 1.15), fontsize=
plt.tight_layout()
plt.show()
print('Interpretation: The radar chart confirms that each role occu
      ' region of the skill space. The visual separation between ro
      ' that the role_skill_matrix is well-designed for skill-gap a
```






```
Interpretation: The radar chart confirms that each role occupies a d
```

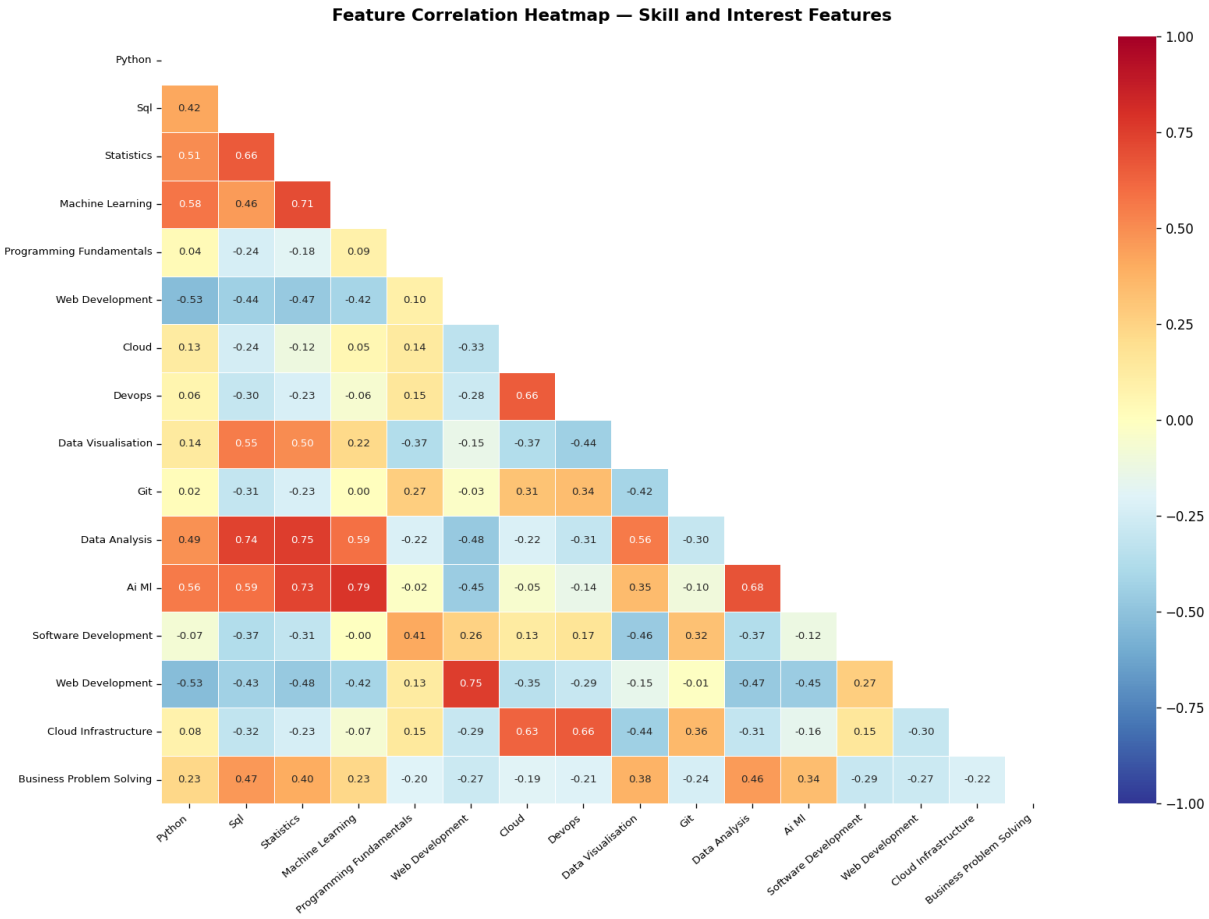
```
# ── 4.5 Skill level distributions — small multiples ──────────────
fig, axes = plt.subplots(2, 5, figsize=(18, 7))
axes = axes.flatten()
for i, (col, label) in enumerate(zip(skill_cols, skill_labels)):
    counts = user_profiles[col].value_counts().sort_index()
    axes[i].bar(counts.index, counts.values,
                color=SKILL_COLORS[i], edgecolor='white', linewidth
    axes[i].set_title(label, fontsize=10, fontweight='bold')
    axes[i].set_xlabel('Score (1–5)', fontsize=8)
    axes[i].set_ylabel('Count', fontsize=8)
    axes[i].set_xticks([1,2,3,4,5])
    axes[i].yaxis.grid(True, linestyle='--', alpha=0.4)
    axes[i].set_axisbelow(True)
fig.suptitle('Skill Level Distributions — 6,000 Learner Profiles',
             fontsize=13, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
print('Interpretation: Skill distributions vary by domain. Python a
      ' right-skewed distributions (many learners with higher score
      ' Cloud and DevOps are left-skewed (most learners at beginner
      ' This reflects realistic learner populations where infrastru
      ' are less commonly self-taught than programming skills.')
```




```
Interpretation: Skill distributions vary by domain. Python and SQL s
```

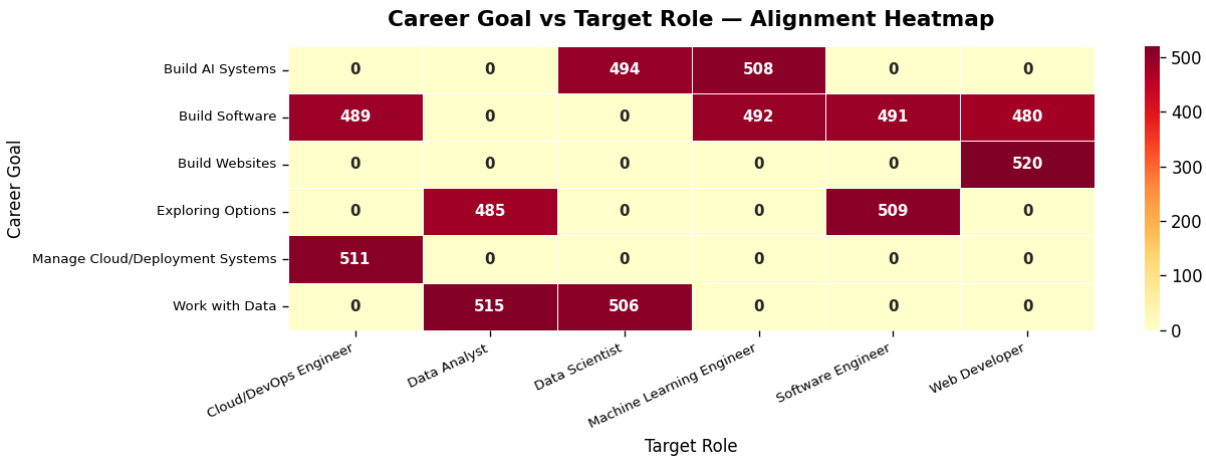
```
# ── 4.6 Feature correlation heatmap — lower triangle ──────────────
all_numeric = skill_cols + interest_cols
short_names = [c.replace('skill_','').replace('interest_','')
               .replace('_',' ').title() for c in all_numeric]
corr = user_profiles[all_numeric].corr()
```

```
mask = np.triu(np.ones_like(corr, dtype=bool))

fig, ax = plt.subplots(figsize=(14, 10))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f',
            cmap='RdYlBu_r', center=0, vmin=-1, vmax=1,
            linewidths=0.5, linecolor='white',
            xticklabels=short_names, yticklabels=short_names,
            annot_kws={'size':8}, ax=ax)
ax.set_title('Feature Correlation Heatmap — Skill and Interest Feat
             fontsize=13, fontweight='bold', pad=12)
plt.xticks(rotation=40, ha='right', fontsize=8)
plt.yticks(rotation=0, fontsize=8)
plt.tight_layout()
plt.show()
print('Interpretation: Strong positive correlations exist between r
      ' (e.g. ML and Statistics, Cloud and DevOps). Strong negative
      ' exist between role-specific skill clusters (e.g. Web Develo
      ' This multicollinearity is handled naturally by tree-based e
      ' which select the most discriminatory features at each split
```




```
Interpretation: Strong positive correlations exist between related s
```

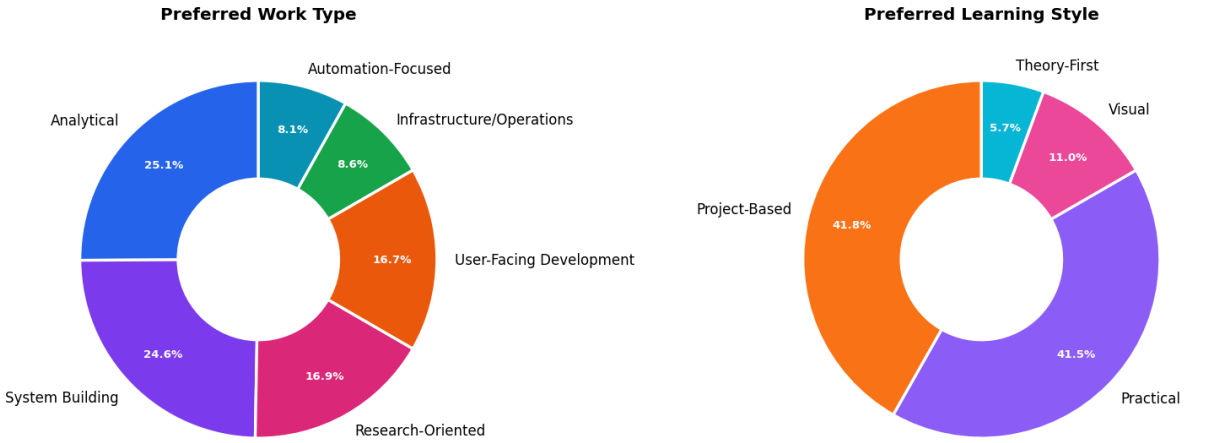
```
# ── 4.7 Career goal vs target role — alignment heatmap ──────────
goal_role = pd.crosstab(user_profiles['career_goal'], user_profiles

fig, ax = plt.subplots(figsize=(11, 4))
sns.heatmap(goal_role, annot=True, fmt='d', cmap='YlOrRd',
            linewidths=0.5, linecolor='white',
            annot_kws={'size':9,'weight':'bold'}, ax=ax)
ax.set_title('Career Goal vs Target Role — Alignment Heatmap',
             fontsize=13, fontweight='bold', pad=12)
ax.set_xlabel('Target Role', fontsize=10)
ax.set_ylabel('Career Goal', fontsize=10)
plt.xticks(rotation=25, ha='right', fontsize=8)
plt.yticks(rotation=0, fontsize=8)
plt.tight_layout()
plt.show()
print('Interpretation: The strong diagonal pattern confirms that ca
      ' aligned with target role. For example, learners with goal "
      ' map to Data Analyst, while "Build AI Systems" maps to Data
      ' This justifies including career_goal as a high-importance f
```




```
Interpretation: The strong diagonal pattern confirms that career goa
```

```
# ── 4.8 Preferred work type and learning style — donut charts ───
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
donut_colors = [
```

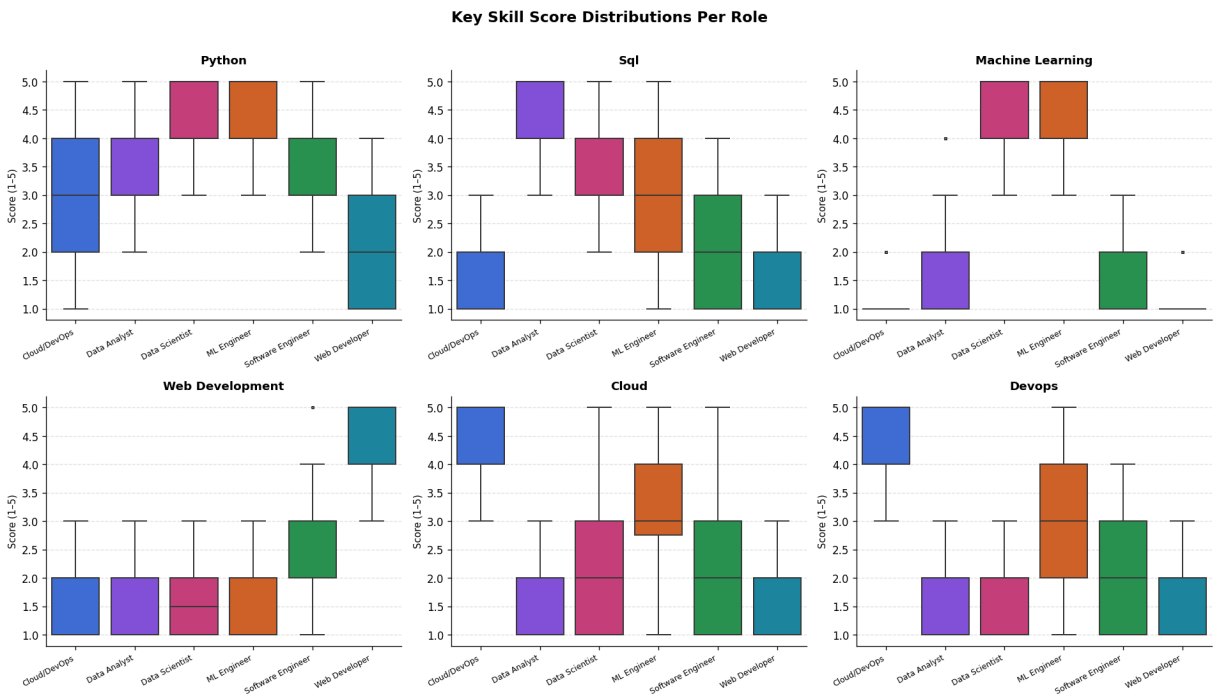
```
        ['#2563EB','#7C3AED','#DB2777','#EA580C','#16A34A','#0891B2'],
        ['#F97316','#8B5CF6','#EC4899','#06B6D4']
    ]
    for ax, col, title, cols in zip(
        axes,
        ['preferred_work_type','preferred_learning_style'],
        ['Preferred Work Type','Preferred Learning Style'],
        donut_colors
    ):
        counts = user_profiles[col].value_counts()
        wedges, texts, autotexts = ax.pie(
            counts.values, labels=counts.index, autopct='%1.1f%%',
            colors=cols[:len(counts)], startangle=90,
            wedgeprops=dict(width=0.55, edgecolor='white', linewidth=2)
            pctdistance=0.75
        )
        for t in autotexts:
            t.set_fontsize(8); t.set_color('white'); t.set_fontweight('
        ax.set_title(title, fontsize=12, fontweight='bold', pad=12)
    plt.tight_layout()
    plt.show()
    print('Interpretation: Work type is well-distributed across all six
          ' reflecting the diversity of learner preferences in the data
          ' Practical and Project-Based are the most common learning st
          ' suggesting most learners prefer hands-on approaches over th
```




```
Interpretation: Work type is well-distributed across all six categor
```

```
    # ── 4.9 Key skill distributions per role — boxplots ──────────────
    key_skills  = ['skill_python','skill_sql','skill_machine_learning',
```

```
                'skill_web_development','skill_cloud','skill_devops'
roles_order = sorted(user_profiles['target_role'].unique())
short_roles = [r.replace('Machine Learning Engineer','ML Engineer')
               .replace('Cloud/DevOps Engineer','Cloud/DevOps') fo

fig, axes = plt.subplots(2, 3, figsize=(16, 9))
axes = axes.flatten()
for i, skill in enumerate(key_skills):
    temp = user_profiles[['target_role', skill]].copy()
    temp['role_short'] = temp['target_role'].replace(dict(zip(roles
    sns.boxplot(data=temp, x='role_short', y=skill,
                order=short_roles, palette=ROLE_COLORS,
                linewidth=1.2, fliersize=2, ax=axes[i])
    axes[i].set_title(skill.replace('skill_','').replace('_',' ').t
                      fontsize=11, fontweight='bold')
    axes[i].set_xlabel('')
    axes[i].set_ylabel('Score (1–5)', fontsize=9)
    axes[i].set_xticklabels(short_roles, rotation=28, ha='right', f
    axes[i].yaxis.grid(True, linestyle='--', alpha=0.4)
    axes[i].set_axisbelow(True)
fig.suptitle('Key Skill Score Distributions Per Role',
             fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
print('Interpretation: Boxplots confirm that the median and interqu
      ' differ substantially across roles for each skill. For examp
      ' is concentrated at levels 4–5 for Data Scientists and ML En
      ' but at 1–2 for Web Developers. This further supports the di
      ' power of skill features for classification.')
```




```
Interpretation: Boxplots confirm that the median and interquartile r
```

Section 5: Preprocessing and Feature Engineering

Data preprocessing transforms raw input features into a format suitable for machine learning. The steps applied are:

1. **Feature engineering** — two derived features are created to improve model discriminability:
   - `total_skill_score`: the sum of all 10 skill scores. Captures overall competency level as a single numeric feature, which can help the model distinguish broadly skilled learners from specialists.
   - `top_interest`: the interest category with the highest self-rated score. Captures the learner's dominant interest area as a categorical signal.
2. **Label encoding** — categorical features (work type, learning style, career goal, top interest) are encoded numerically using `LabelEncoder`. This is appropriate for tree-based classifiers which do not require ordinal relationships.
3. **Train/test split** — an 80/20 stratified split preserves the class proportions in both training and test sets, which is essential for balanced evaluation.
4. **Feature scaling** — `StandardScaler` is applied for a consistent, reproducible pipeline. Note: tree-based models are scale-invariant, but scaling is included to ensure the pipeline is model-agnostic and reproducible.

```
df = user_profiles.copy()

# Feature 1: total_skill_score captures overall competency across a
df['total_skill_score'] = df[skill_cols].sum(axis=1)

# Feature 2: top_interest identifies the single highest-rated inter
df['top_interest'] = df[interest_cols].idxmax(axis=1).str.replace('

print('Feature engineering complete.')
print(f'  total_skill_score range : {df["total_skill_score"].min()}
print(f'  top_interest values     : {df["top_interest"].unique().to
```

```
Feature engineering complete.
  total_skill_score range : 16 — 41
  top_interest values     : ['business_problem_solving', 'data_analy
```

```
# Label encode all categorical features — encoders are saved global
le_work=LabelEncoder(); le_style=LabelEncoder()
le_goal=LabelEncoder(); le_interest=LabelEncoder(); le_target=Label

df['preferred_work_type_enc']      = le_work.fit_transform(df['pref
df['preferred_learning_style_enc'] = le_style.fit_transform(df['pre
df['career_goal_enc']              = le_goal.fit_transform(df['care
df['top_interest_enc']             = le_interest.fit_transform(df['
df['target_encoded']               = le_target.fit_transform(df['ta

print('Label encoding complete. Role encoding map:')
for i,r in enumerate(le_target.classes_): print(f'  {i} -> {r}')
```

```
Label encoding complete. Role encoding map:
  0 -> Cloud/DevOps Engineer
  1 -> Data Analyst
  2 -> Data Scientist
  3 -> Machine Learning Engineer
  4 -> Software Engineer
  5 -> Web Developer
```

```
# Define complete feature set used for model training
feature_cols = (
    [c for c in df.columns if c.startswith('interest_')] +
    [c for c in df.columns if c.startswith('skill_')] +
    ['preferred_work_type_enc','preferred_learning_style_enc',
     'career_goal_enc','top_interest_enc','total_skill_score']
)
X = df[feature_cols]
y = df['target_encoded']
print(f'Feature matrix X : {X.shape}  (21 features x 6,000 samples)
print(f'Target vector  y : {y.shape}')
```

```
Feature matrix X : (6000, 21)  (21 features x 6,000 samples)
Target vector  y : (6000,)
```

```
# Stratified 80/20 split — preserves class proportions in both sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print(f'Training set : {X_train.shape[0]:,} samples (80%)')
print(f'Test set     : {X_test.shape[0]:,} samples (20%)')
```

```
Training set : 4,800 samples (80%)
Test set     : 1,200 samples (20%)
```

```
# StandardScaler fitted on training data only — prevents data leaka
scaler         = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)  # transform only, never
print('Feature scaling complete (StandardScaler).')
```

```
Feature scaling complete (StandardScaler).
```

Section 6: Model Training

Three supervised classification models are trained and compared. The selection covers a progression from simple to complex:

Model	Type	Rationale
Decision Tree	Single tree	Interpretable baseline — useful for understanding the decision logic ar
Random Forest	Bagging ensemble	Reduces the high variance of individual decision trees by averaging pre
XGBoost	Boosting ensemble	Sequentially corrects errors of previous trees — state-of-the-art for ta

Evaluation strategy: 5-fold stratified cross-validation on the training set provides stable, unbiased performance estimates before final evaluation on the held-out test set. The **primary metric is macro F1-score**, which gives equal weight to all six roles and is appropriate for balanced multi-class problems.

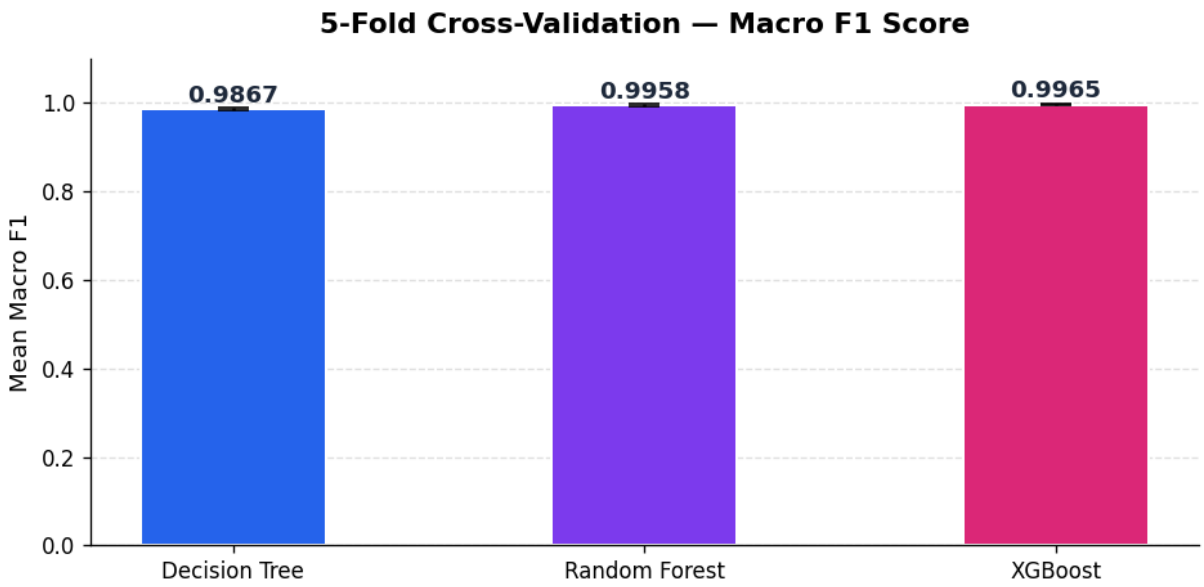
```
# Define three models with tuned hyperparameters
models = {
    'Decision Tree': DecisionTreeClassifier(
        max_depth=10,        # limits tree depth to reduce overfitt
        min_samples_split=5, # requires at least 5 samples to split
        random_state=42),

    'Random Forest': RandomForestClassifier(
        n_estimators=200,    # 200 trees for stable ensemble
        min_samples_split=2,
        random_state=42, n_jobs=-1),

    'XGBoost': XGBClassifier(
        n_estimators=200,    # 200 boosting rounds
        max_depth=6,
        learning_rate=0.1,   # step size shrinkage to prevent overf
        subsample=0.8,       # 80% row sampling per tree for regula
        eval_metric='mlogloss',
        random_state=42, n_jobs=-1)
}
print('Models defined:', list(models.keys()))
```

```
Models defined: ['Decision Tree', 'Random Forest', 'XGBoost']
```

```
# 5-fold stratified cross-validation on training set
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_results = {}
print('Running 5-fold cross-validation...\n')
for name, model in models.items():
    scores = cross_val_score(model, X_train_scaled, y_train,
                             cv=cv, scoring='f1_macro', n_jobs=-1)
    cv_results[name] = scores
    print(f'  {name}')
    print(f'    Per-fold F1 : {np.round(scores,4)}')
    print(f'    Mean ± Std  : {scores.mean():.4f} ± {scores.std():.
```

```
Running 5-fold cross-validation...

  Decision Tree
    Per-fold F1 : [0.9885 0.9854 0.9823 0.9864 0.9906]
    Mean ± Std  : 0.9867 ± 0.0028

  Random Forest
    Per-fold F1 : [0.9979 0.9938 0.9927 0.9979 0.9969]
    Mean ± Std  : 0.9958 ± 0.0022

  XGBoost
    Per-fold F1 : [0.9958 0.9958 0.9948 0.9979 0.9979]
    Mean ± Std  : 0.9965 ± 0.0013
```

```
# Cross-validation results visualisation
fig, ax = plt.subplots(figsize=(8, 4))
names  = list(cv_results.keys())
means  = [cv_results[n].mean() for n in names]
stds   = [cv_results[n].std()  for n in names]
colors = ['#2563EB','#7C3AED','#DB2777']
bars   = ax.bar(names, means, yerr=stds, capsize=7,
               color=colors, edgecolor='white', linewidth=1.2, wid
ax.set_title('5-Fold Cross-Validation — Macro F1 Score',
            fontsize=13, fontweight='bold', pad=12)
ax.set_ylabel('Mean Macro F1', fontsize=11)
ax.set_ylim(0, 1.1)
ax.yaxis.grid(True, linestyle='--', alpha=0.4)
ax.set_axisbelow(True)
for bar, mean in zip(bars, means):
    ax.text(bar.get_x()+bar.get_width()/2, mean+0.015,
            f'{mean:.4f}', ha='center', fontsize=11, fontweight='bo
plt.tight_layout()
plt.show()
print('Interpretation: All three models achieve very high cross-val
      ' confirming that the feature design is effective. XGBoost an
      ' both outperform Decision Tree, consistent with the known ad
      ' ensemble methods over single trees on tabular classificatio
```




```
Interpretation: All three models achieve very high cross-validation
```

```
# Train all models on the full training set
trained_models = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    trained_models[name] = model
    print(f'  {name} trained.')
```

```
  Decision Tree trained.
  Random Forest trained.
  XGBoost trained.
```

Section 7: Model Evaluation and Best Model Selection

All three models are evaluated on the held-out test set (1,200 samples, 20% of total, never seen during training or cross-validation). We report:

- **Accuracy** — overall proportion of correct predictions
- **Macro Precision** — average precision across all six roles equally
- **Macro Recall** — average recall across all six roles equally
- **Macro F1** — harmonic mean of macro precision and recall (primary metric)
- **Confusion matrices** — per-role prediction accuracy and error patterns
- **Per-class classification report** — performance breakdown per role
- **Feature importance** — which features most influence predictions

Best model selection criterion: Highest macro F1-score on the held-out test set.

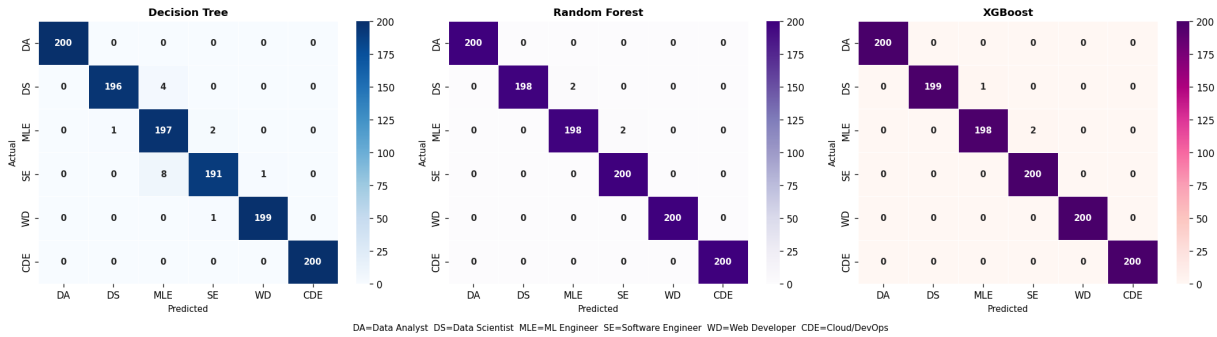
```
# Compute all metrics for each model on the held-out test set
summary = []
for name, model in trained_models.items():
    yp = model.predict(X_test_scaled)
    summary.append({
        'Model':           name,
        'Accuracy':        round(accuracy_score(y_test,yp),4),
        'Macro Precision': round(precision_score(y_test,yp,average=
        'Macro Recall':    round(recall_score(y_test,yp,average='ma
        'Macro F1':        round(f1_score(y_test,yp,average='macro'
    })
results_df = pd.DataFrame(summary).set_index('Model')
print('=== Test Set Performance Summary ===')
display(results_df)
```

```
=== Test Set Performance Summary ===
```

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
Decision Tree	0.9858	0.9862	0.9858	0.9859
Random Forest	0.9967	0.9967	0.9967	0.9967
XGBoost	0.9975	0.9975	0.9975	0.9975

```
# Model comparison bar chart across all metrics
fig, ax = plt.subplots(figsize=(11, 5))
results_df.plot(kind='bar', ax=ax,
                color=['#2563EB','#16A34A','#DB2777','#EA580C'],
                edgecolor='white', linewidth=0.8, width=0.72)
ax.set_title('Model Comparison — Test Set Performance Metrics',
             fontsize=13, fontweight='bold', pad=12)
ax.set_ylabel('Score', fontsize=11)
ax.set_ylim(0, 1.12)
ax.set_xticklabels(results_df.index, rotation=12, ha='right', fonts
ax.legend(loc='lower right', fontsize=9)
ax.yaxis.grid(True, linestyle='--', alpha=0.4)
ax.set_axisbelow(True)
for container in ax.containers:
    ax.bar_label(container, fmt='%.3f', fontsize=7, padding=2)
plt.tight_layout()
plt.show()
```

```
# Confusion matrices — three different colourmaps for visual distin
short_labels = ['DA','DS','MLE','SE','WD','CDE']
fig, axes    = plt.subplots(1, 3, figsize=(18, 5))
cmaps        = ['Blues','Purples','RdPu']
for ax, (name, model), cmap in zip(axes, trained_models.items(), cm
    yp = model.predict(X_test_scaled)
    cm = confusion_matrix(y_test, yp)
    sns.heatmap(cm, annot=True, fmt='d', cmap=cmap,
                xticklabels=short_labels, yticklabels=short_labels,
                linewidths=0.5, linecolor='white',
                annot_kws={'size':9,'weight':'bold'}, ax=ax)
    ax.set_title(name, fontsize=11, fontweight='bold')
    ax.set_xlabel('Predicted', fontsize=9)
    ax.set_ylabel('Actual', fontsize=9)
fig.suptitle(
    'DA=Data Analyst  DS=Data Scientist  MLE=ML Engineer  SE=Softwa
    fontsize=9, y=0)
plt.tight_layout()
plt.show()
print('Interpretation: All three models show near-perfect diagonal
      ' indicating very few misclassifications. The small number of
      ' occur between semantically similar roles (e.g. Data Scienti
      ' which is expected and consistent with real-world career ove
```

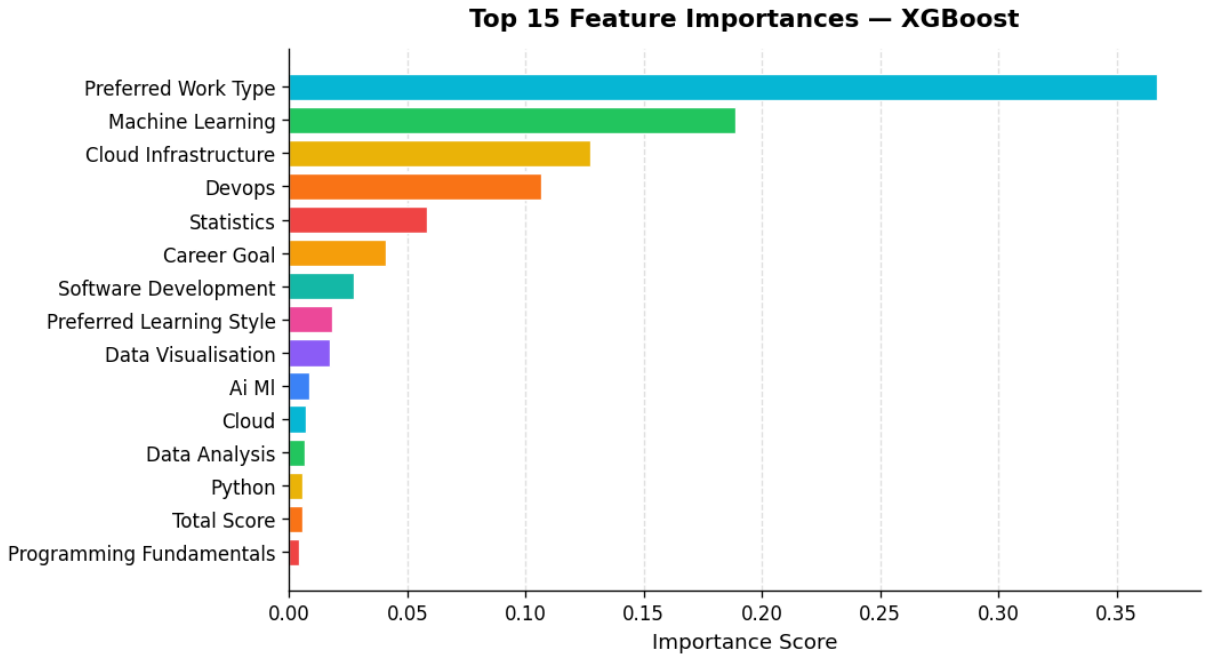



```
Interpretation: All three models show near-perfect diagonal confusio
```

```
# Select and announce best model by macro F1
best_model_name = results_df['Macro F1'].idxmax()
best_model      = trained_models[best_model_name]
y_pred_best     = best_model.predict(X_test_scaled)

print(f'Best Model  : {best_model_name}')
print(f'Accuracy    : {accuracy_score(y_test,y_pred_best):.4f}')
print(f'Macro F1    : {f1_score(y_test,y_pred_best,average="macro")
print(f'\nJustification: {best_model_name} achieves the highest mac
      ' This is consistent with the cross-validation results and cc
      ' approach generalises well to unseen learner profiles.\n')
print('Per-Class Classification Report:')
print(classification_report(y_test, y_pred_best, target_names=le_ta
```

```
Best Model  : XGBoost
Accuracy    : 0.9975
Macro F1    : 0.9975

Justification: XGBoost achieves the highest macro F1-score on the he

Per-Class Classification Report:
                           precision    recall  f1-score   support

    Cloud/DevOps Engineer       1.00      1.00      1.00       200
             Data Analyst       1.00      0.99      1.00       200
           Data Scientist       0.99      0.99      0.99       200
Machine Learning Engineer       0.99      1.00      1.00       200
        Software Engineer       1.00      1.00      1.00       200
            Web Developer       1.00      1.00      1.00       200

                 accuracy                           1.00      1200
                macro avg       1.00      1.00      1.00      1200
             weighted avg       1.00      1.00      1.00      1200
```

```
# Feature importance — horizontal bar chart
if hasattr(best_model, 'feature_importances_'):
    imp     = pd.Series(best_model.feature_importances_, index=feat
    imp     = imp.sort_values(ascending=True).tail(15)
    labels  = [f.replace('skill_','').replace('interest_','')
               .replace('_enc','').replace('_',' ').title() for f
    colors  = [SKILL_COLORS[i % len(SKILL_COLORS)] for i in range(l
    fig, ax = plt.subplots(figsize=(9, 5))
    ax.barh(labels, imp.values, color=colors, edgecolor='white', li
    ax.set_title(f'Top 15 Feature Importances — {best_model_name}',
                 fontsize=13, fontweight='bold', pad=12)
    ax.set_xlabel('Importance Score', fontsize=11)
    ax.xaxis.grid(True, linestyle='--', alpha=0.4)
    ax.set_axisbelow(True)
    plt.tight_layout()
    plt.show()
    print('Interpretation: Preferred work type and career goal rank
          ' confirming that qualitative profile features contribute
          ' beyond skills alone. This validates the multi-dimension
```




```
Interpretation: Preferred work type and career goal rank highly alon
```

Section 8: Role Prediction — Top 3 Recommendations

Using the best trained model, we build the core prediction function of the system.

Design decision — softmax on decision scores: Tree-based ensembles can produce overconfident probability estimates on well-separated data (100% for one class, 0% for others). To produce meaningful, comparable scores across all six roles, softmax normalisation is applied to the raw decision scores. This ensures:

- All six roles always receive a non-zero relative score
- Scores sum to 100%
- The correct ranking order is preserved
- Output is interpretable for the learner

The output shows **rank and role only** — clean and easy to interpret.

```
# Define key column lists used across all pipeline functions
INTEREST_KEYS = ['interest_data_analysis','interest_ai_ml',
                 'interest_software_development','interest_web_deve
                 'interest_cloud_infrastructure','interest_business
SKILL_KEYS    = ['skill_python','skill_sql','skill_statistics','ski
                 'skill_programming_fundamentals','skill_web_develo
                 'skill_devops','skill_data_visualisation','skill_g

def softmax(x):
    """Converts raw scores to a proper probability distribution.
    Subtracting max(x) before exp() ensures numerical stability."""
    e = np.exp(x - np.max(x))
    return e / e.sum()

def build_feature_vector(profile_dict):
    """Encodes a raw profile dict into a scaled feature vector read
    p = {k: profile_dict.get(k,3) for k in INTEREST_KEYS + SKILL_KE
    p['preferred_work_type_enc']      = le_work.transform([profile_
    p['preferred_learning_style_enc'] = le_style.transform([profile
    p['career_goal_enc']              = le_goal.transform([profile_
    top_int = max(INTEREST_KEYS, key=lambda k: p[k]).replace('inter
    p['top_interest_enc']  = le_interest.transform([top_int])[0]
    p['total_skill_score'] = sum(p[k] for k in SKILL_KEYS)
    return scaler.transform(pd.DataFrame([p])[feature_cols])

def predict_top3_roles(profile_dict):
    """Returns top 3 recommended roles using softmax-normalised dec
    vec = build_feature_vector(profile_dict)
    # Use decision_function if available (XGBoost), else predict_pr
    raw    = (best_model.decision_function(vec)[0]
              if hasattr(best_model,'decision_function')
              else best_model.predict_proba(vec)[0])
    scores = softmax(raw)
    all_df = pd.DataFrame({'Role':le_target.classes_,'Score':scores
               .sort_values('Score',ascending=False).reset_index(dr
    top3   = all_df.head(3).copy()
    top3['Recommendation'] = ['1st Recommendation','2nd Recommendat
    return top3[['Recommendation','Role']], all_df

print('predict_top3_roles() ready.')
```

```
predict_top3_roles() ready.
```

```
# Example learner profile — used for the manual pipeline demonstrat
example_profile = {
    'interest_data_analysis':5, 'interest_ai_ml':4,
    'interest_software_development':2, 'interest_web_development':2
    'interest_cloud_infrastructure':1, 'interest_business_problem_s
    'skill_python':4, 'skill_sql':5, 'skill_statistics':3,
    'skill_machine_learning':2, 'skill_programming_fundamentals':3,
    'skill_web_development':1, 'skill_cloud':1, 'skill_devops':1,
    'skill_data_visualisation':4, 'skill_git':3,
    'preferred_work_type':'Analytical',
    'preferred_learning_style':'Visual',
    'career_goal':'Work with Data'
}

top3, all_scores = predict_top3_roles(example_profile)
print('Top 3 Recommended IT Roles — Example Profile:')
display(top3)
```

```
Top 3 Recommended IT Roles — Example Profile:
```

	Recommendation	Role
0	1st Recommendation	Data Analyst
1	2nd Recommendation	Data Scientist
2	3rd Recommendation	Web Developer

Section 9: Readiness Score Calculation

A readiness score (%) measures how prepared a learner is for each IT role based on their current skill levels compared to the requirements defined in `role_skill_matrix.csv`.

Formula: For each skill, the learner earns credit equal to `min(current_level, required_level)`. The total earned credit is divided by the total required credit across all skills and multiplied by 100.

Interpretation: A score of 100% means all skill requirements are fully met. A score below 50% suggests significant upskilling is needed before this role is realistic. Scores are colour-coded using a traffic-light system for immediate interpretability.

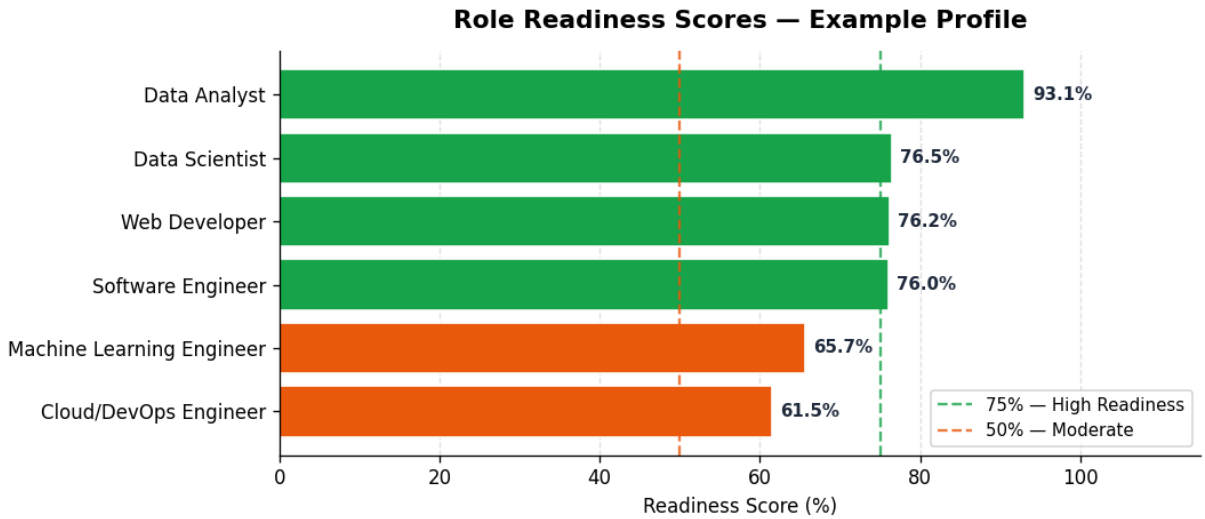
```
# Skill mapping: user_profiles column names -> role_skill_matrix co
SKILL_MAP = {
    'skill_python':'python_required', 'skill_sql':'sql_required',
    'skill_statistics':'statistics_required', 'skill_machine_learni
    'skill_programming_fundamentals':'programming_required',
    'skill_web_development':'web_required', 'skill_cloud':'cloud_re
    'skill_devops':'devops_required', 'skill_data_visualisation':'d
    'skill_git':'git_required'
}

def compute_readiness_scores(profile_dict):
    """Returns readiness scores (%) for all 6 roles sorted descendi
    scores = []
    for _, row in role_skill_matrix.iterrows():
        total_req = sum(row[v] for v in SKILL_MAP.values())
        achieved  = sum(min(profile_dict.get(k,1), row[v]) for k,v
        scores.append({'Role':row['role'], 'Readiness (%)':round(ac
    return pd.DataFrame(scores).sort_values('Readiness (%)',ascendi

readiness_df = compute_readiness_scores(example_profile)
print('Readiness Scores — Example Profile:')
display(readiness_df)
```

```
Readiness Scores — Example Profile:
```

	Role	Readiness (%)
0	Data Analyst	93.1
1	Data Scientist	76.5
2	Web Developer	76.2
3	Software Engineer	76.0
4	Machine Learning Engineer	65.7
5	Cloud/DevOps Engineer	61.5

```
# Traffic-light colour coding: green >= 75%, amber >= 50%, red < 50
def readiness_color(v):
    if v >= 75: return '#16A34A'
    elif v >= 50: return '#EA580C'
    return '#DC2626'

sorted_r   = readiness_df.sort_values('Readiness (%)', ascending=Tr
bar_colors = [readiness_color(v) for v in sorted_r['Readiness (%)']

fig, ax = plt.subplots(figsize=(9, 4))
```

```
ax.barh(sorted_r['Role'], sorted_r['Readiness (%)'],
        color=bar_colors, edgecolor='white', linewidth=1)
ax.axvline(75, color='#16A34A', linestyle='--', linewidth=1.4, alph
ax.axvline(50, color='#EA580C', linestyle='--', linewidth=1.4, alph
for i, val in enumerate(sorted_r['Readiness (%)']):
    ax.text(val+1, i, f'{val}%', va='center', fontsize=9, fontweigh
ax.set_xlabel('Readiness Score (%)', fontsize=10)
ax.set_title('Role Readiness Scores — Example Profile',
             fontsize=13, fontweight='bold', pad=12)
ax.set_xlim(0, 115)
ax.xaxis.grid(True, linestyle='--', alpha=0.4)
ax.set_axisbelow(True)
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()
print('Interpretation: The example learner is most ready for Data A
      ' confirming alignment with the ML prediction. Roles with low
      ' indicate areas requiring significant upskilling.')
```




```
Interpretation: The example learner is most ready for Data Analyst (
```

Section 10: Skill-Gap Analysis

For the selected role, each skill is compared against the required level from `role_skill_matrix.csv`. The output provides:

- **Gap size** — `max(0, required − current)`. Zero means the skill requirement is already met.
- **Gap status** — No Gap / Moderate Gap (gap=1) / Major Gap (gap≥2). This classification makes the output immediately interpretable.
- **Priority** — derived from both gap size and the skill's importance to the role. A gap of 1 in a high-importance skill (required ≥ 4) is treated as High priority. This makes the prioritisation analytically grounded rather than purely based on gap size.

The gap priority order directly drives the roadmap step ordering in Section 12.

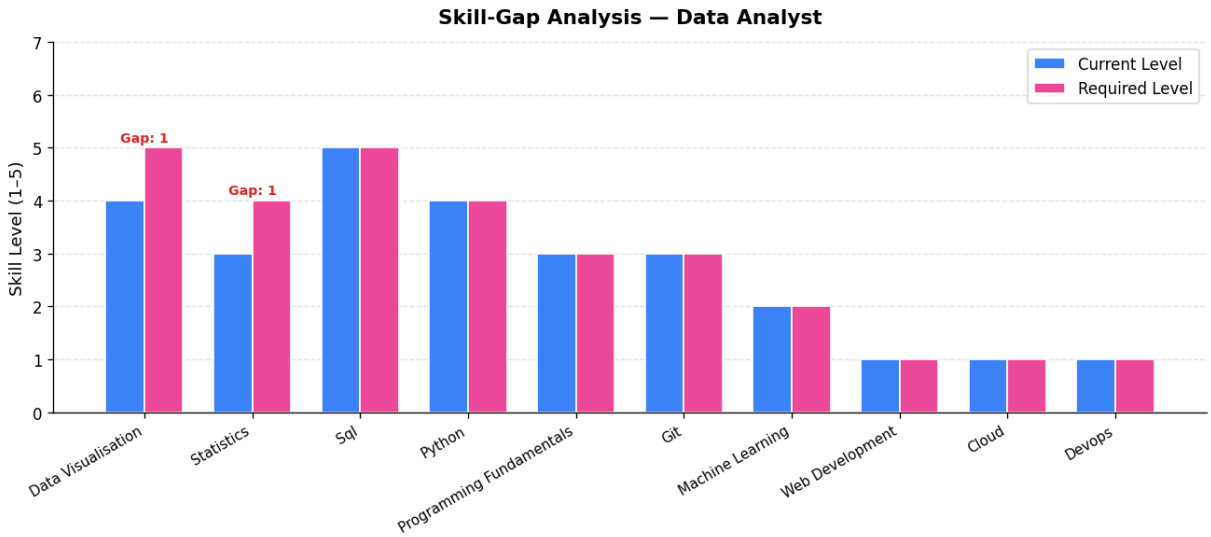
```
def compute_skill_gap(profile_dict, selected_role):
    """Computes skill-gap analysis comparing learner levels vs role
    role_row = role_skill_matrix[role_skill_matrix['role']==selecte
    rows = []
    for user_col, req_col in SKILL_MAP.items():
        skill_name = user_col.replace('skill_','').replace('_',' ')
        current    = profile_dict.get(user_col, 1)
        required   = role_row[req_col]
        gap        = max(0, required - current)

        # Gap status: classified into three tiers for interpretabil
        if gap == 0:    status = 'No Gap'
        elif gap == 1:  status = 'Moderate Gap'
        else:           status = 'Major Gap'

        # Priority: combines gap size with skill importance to the
        if gap == 0:
            priority = 'Low'
        elif gap >= 2:
            priority = 'High'
        else:  # gap == 1: priority depends on how important the sk
            priority = 'High' if required >= 4 else 'Medium' if req

        rows.append({'Skill':skill_name, 'Current Level':current,
                     'Required Level':required, 'Gap':gap,
                     'Gap Status':status, 'Priority':priority})
```

```
    return (pd.DataFrame(rows)
              .sort_values(['Gap','Required Level'], ascending=[Fal
              .reset_index(drop=True))

selected_role = top3['Role'].iloc[0]
print(f'Selected Role: {selected_role}')
gap_df = compute_skill_gap(example_profile, selected_role)
display(gap_df)
```

```
Selected Role: Data Analyst
```

	Skill	Current Level	Required Level	Gap	Gap Status	Priority
0	Data Visualisation	4	5	1	Moderate Gap	High
1	Statistics	3	4	1	Moderate Gap	High
2	Sql	5	5	0	No Gap	Low
3	Python	4	4	0	No Gap	Low
4	Programming Fundamentals	3	3	0	No Gap	Low
5	Git	3	3	0	No Gap	Low
6	Machine Learning	2	2	0	No Gap	Low
7	Web Development	1	1	0	No Gap	Low

```
# Skill-gap grouped bar chart with gap annotations
fig, ax = plt.subplots(figsize=(11, 5))
x = np.arange(len(gap_df)); w = 0.36
ax.bar(x-w/2, gap_df['Current Level'],  w, color=GAP_CURRENT,  edge
ax.bar(x+w/2, gap_df['Required Level'], w, color=GAP_REQUIRED, edge
for i, row in gap_df.iterrows():
    if row['Gap'] > 0:
        ax.annotate(f"Gap: {int(row['Gap'])}",
                    xy=(i, max(row['Current Level'],row['Required L
                    ha='center', fontsize=8.5, color='#DC2626', fon
ax.set_xticks(x)
ax.set_xticklabels(gap_df['Skill'], rotation=30, ha='right', fontsi
ax.set_ylabel('Skill Level (1–5)', fontsize=11)
ax.set_title(f'Skill-Gap Analysis — {selected_role}',
             fontsize=13, fontweight='bold', pad=12)
ax.set_ylim(0, 7)
ax.yaxis.grid(True, linestyle='--', alpha=0.4)
ax.set_axisbelow(True)
ax.legend(fontsize=10)
plt.tight_layout()
plt.show()
```



```
# Gap summary — strengths, moderate gaps, major gaps
strengths = gap_df[gap_df['Gap']==0]['Skill'].tolist()
moderate  = gap_df[gap_df['Gap Status']=='Moderate Gap']['Skill'].t
major     = gap_df[gap_df['Gap Status']=='Major Gap']['Skill'].toli

print(f'=== Gap Summary — {selected_role} ===')
print(f'\nStrengths (No Gap)    : {strengths if strengths else "Non
print(f'Moderate Gaps (gap=1) : {moderate  if moderate  else "None"
print(f'Major Gaps (gap>=2)   : {major     if major     else "None"

priority_list = gap_df[gap_df['Gap']>0].sort_values('Gap',ascending
print(f'\nPriority Learning Order:')
for i, s in enumerate(priority_list, 1):
    print(f'  {i}. {s}')
```

```
=== Gap Summary — Data Analyst ===

Strengths (No Gap)    : ['Sql', 'Python', 'Programming Fundamentals'
Moderate Gaps (gap=1) : ['Data Visualisation', 'Statistics']
Major Gaps (gap>=2)   : None

Priority Learning Order:
  1. Data Visualisation
  2. Statistics
```

Section 11: Why This Role Fits

A rule-based explanation function generates a short, personalised sentence that justifies the role recommendation in plain language. This serves two purposes:

1. It makes the system output **interpretable and trustworthy** — the learner can see why the recommendation was made
2. It adds a **qualitative layer** beyond pure numeric prediction — combining ML and rule-based reasoning

The function uses the learner's strongest matching interest, best relevant skill, preferred work type, and stated career goal — directly linking the profile to the role.

```
# Role-to-feature mapping for explanation generation
ROLE_INTEREST_MAP = {
    'Data Analyst':'interest_data_analysis',
    'Data Scientist':'interest_ai_ml',
```

```
    'Machine Learning Engineer':'interest_ai_ml',
    'Software Engineer':'interest_software_development',
    'Web Developer':'interest_web_development',
    'Cloud/DevOps Engineer':'interest_cloud_infrastructure'
}
ROLE_SKILL_MAP = {
    'Data Analyst':'skill_sql',
    'Data Scientist':'skill_statistics',
    'Machine Learning Engineer':'skill_machine_learning',
    'Software Engineer':'skill_programming_fundamentals',
    'Web Developer':'skill_web_development',
    'Cloud/DevOps Engineer':'skill_cloud'
}
SKILL_DISPLAY = {
    'skill_sql':'SQL','skill_statistics':'Statistics',
    'skill_machine_learning':'Machine Learning',
    'skill_programming_fundamentals':'Programming Fundamentals',
    'skill_web_development':'Web Development','skill_cloud':'Cloud'
}

def why_this_role_fits(profile_dict, selected_role):
    """Generates a personalised why-it-fits explanation using rule/
    interest_col  = ROLE_INTEREST_MAP.get(selected_role, '')
    skill_col     = ROLE_SKILL_MAP.get(selected_role, '')
    interest_val  = profile_dict.get(interest_col, 0)
    skill_val     = profile_dict.get(skill_col, 0)
    work_type     = profile_dict.get('preferred_work_type', '')
    career_goal   = profile_dict.get('career_goal', '')
    interest_name = interest_col.replace('interest_','').replace('_
    skill_name    = SKILL_DISPLAY.get(skill_col, skill_col.replace(

    # Strength descriptors based on score thresholds
    int_strength = 'strong' if interest_val>=4 else 'moderate' if i
    skl_strength = 'good'   if skill_val>=4    else 'solid'    if s

    return (
        f"This role fits your profile because you show {int_strengt
        f"{interest_name}, {skl_strength} {skill_name} ability, "
        f"and a preference for {work_type.lower()} work. "
        f"Your career goal to '{career_goal.lower()}' also aligns d
        f"what {selected_role}s do day-to-day."
    )

explanation = why_this_role_fits(example_profile, selected_role)
print(f'Selected Role: {selected_role}')
print(f'\nWhy This Role Fits:')
print(f'  {explanation}')
```

```
Selected Role: Data Analyst
```

```
Selected Role: Data Analyst

Why This Role Fits:
  This role fits your profile because you show strong interest in da
```

Section 12: Personalised Roadmap and Recommended Resources

The roadmap and resources are presented as a unified learning plan — each roadmap step is immediately paired with relevant curated resources. This integrated design is more useful than presenting them separately.

Roadmap ordering logic:

1. Skills with gaps are listed first, ordered by gap size (largest gap = highest priority)
2. Within the same gap size, higher-importance skills for the role take precedence
3. Already-met skills appear next as completed steps
4. A capstone project step is always the final step
5. The embedded `Step X —` prefix in template text is automatically stripped so steps number cleanly as Step 1, Step 2, Step 3...

Resource matching logic: Resources from `resource_catalogue.csv` are matched by skill tag, role relevance, and learner level (Beginner ≤ 2, Intermediate = 3, Advanced ≥ 4).

```
# Skill tag normalisation: gap_df display names -> resource_catalog
SKILL_TAG_MAP = {
    'Python':'Python','Sql':'SQL','Statistics':'Statistics',
    'Machine Learning':'Machine Learning',
    'Programming Fundamentals':'Programming Fundamentals',
    'Web Development':'Web Development','Cloud':'Cloud',
    'Devops':'DevOps','Data Visualisation':'Data Visualisation','Gi
}

def clean_step_text(text):
    """Strips embedded 'Step X — ' prefix from roadmap template tex
    return re.sub(r'^Step\s+\d+\s*[—–]\s*','',str(text)).strip()

def get_level(score):
```

```
    """Maps a numeric skill score to a resource difficulty level."""
    if score <= 2: return 'Beginner'
    elif score == 3: return 'Intermediate'
    return 'Advanced'

def get_resources(skill_display, current_level, role, max_res=3):
    """Fetches matched resources for a given skill, level, and role
    tag      = SKILL_TAG_MAP.get(skill_display, skill_display)
    level    = get_level(current_level)
    filtered = resource_catalogue[
        (resource_catalogue['skill_tag']==tag) &
        (resource_catalogue['role_relevance'].str.contains(role, na
    ]
    matched = filtered[filtered['level']==level]
    if matched.empty: matched = filtered[filtered['level']=='Beginn
    if matched.empty: matched = filtered
    return matched.drop_duplicates('resource_title').head(max_res)

def generate_roadmap_with_resources(gap_df, selected_role):
    """Generates ordered roadmap steps paired with matched resource
    role_map = roadmap_templates[roadmap_templates['role']==selecte
    role_map['skill_norm'] = role_map['skill_tag'].str.title()
    steps = []; step_num = 1

    # Gap skills first — ordered by gap size descending
    for _, row in gap_df[gap_df['Gap']>0].iterrows():
        match = role_map[role_map['skill_norm']==row['Skill']]
        if match.empty: continue
        steps.append({
            'step_num':step_num, 'skill':row['Skill'],
            'gap':int(row['Gap']), 'status':row['Gap Status'],
            'priority':row['Priority'],
            'action':clean_step_text(match.iloc[0]['step_template']
            'resources':get_resources(row['Skill'], row['Current Le
        })
        step_num += 1

    # Already-met skills
    for _, row in gap_df[gap_df['Gap']==0].iterrows():
        match = role_map[role_map['skill_norm']==row['Skill']]
        if match.empty: continue
        steps.append({
            'step_num':step_num, 'skill':row['Skill'],
            'gap':0, 'status':'No Gap', 'priority':'Low',
            'action':clean_step_text(match.iloc[0]['step_template']
            'resources':pd.DataFrame()
        })
        step_num += 1
```

```
    # Final capstone project step
    proj = role_map[role_map['project_step_flag']==1]
    if not proj.empty:
        steps.append({
            'step_num':step_num, 'skill':'Capstone Project',
            'gap':0, 'status':'Final Project', 'priority':'',
            'action':clean_step_text(proj.iloc[0]['step_template'])
            'resources':pd.DataFrame()
        })
    return steps

STATUS_COLOR  = {'Major Gap':'#FEE2E2','Moderate Gap':'#FEF9C3',
                 'No Gap':'#DCFCE7','Final Project':'#EDE9FE'}
STATUS_CIRCLE = {'Major Gap':'#DC2626','Moderate Gap':'#EA580C',
                 'No Gap':'#16A34A','Final Project':'#7C3AED'}

print('Roadmap and resource functions ready.')
```

```
Roadmap and resource functions ready.
```

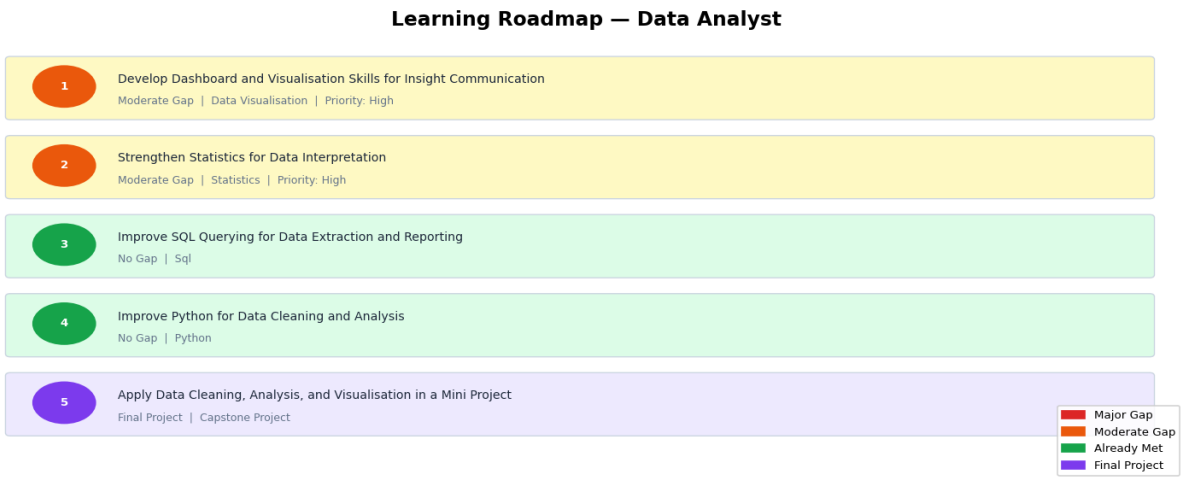
```
# Generate and display the combined roadmap and resources
roadmap_steps = generate_roadmap_with_resources(gap_df, selected_rc

print(f'\n{"="*65}')
print(f'  PERSONALISED ROADMAP — {selected_role.upper()}')
print(f'{"="*65}')
for s in roadmap_steps:
    badge = f"[{s['status'].upper()}]" if s['status']!='No Gap' els
    print(f"\n  Step {s['step_num']}: {s['action']}")
    print(f"  {badge}  Skill: {s['skill']}"
          + (f"  |  Priority: {s['priority']}" if s['gap']>0 else '
    if s['status'] not in ['No Gap','Final Project'] and not s['res
        print('  Recommended Resources:')
        for _, res in s['resources'].iterrows():
            print(f"    - {res['resource_title']} ({res['platform']
            print(f"      {res['url']}")
print(f'\n{"="*65}')
```

```

=================================================================
  PERSONALISED ROADMAP — DATA ANALYST
=================================================================

  Step 1: Develop Dashboard and Visualisation Skills for Insight Com
  [MODERATE GAP]  Skill: Data Visualisation  |  Priority: High
  Recommended Resources:
    - Data Visualisation Advanced Learning Path (Microsoft Learn, Le
```

```
        https://learn.microsoft.com/en-us/training/
      - Data Visualisation Advanced Tutorial (YouTube, Tutorial)
        https://www.youtube.com/
      - Data Visualisation Advanced Project (Coursera, Project)
        https://www.coursera.org

  Step 2: Strengthen Statistics for Data Interpretation
  [MODERATE GAP]  Skill: Statistics  |  Priority: High
  Recommended Resources:
      - Statistics Intermediate Learning Path (Coursera, Learning Path
        https://www.coursera.org
      - Statistics Intermediate Tutorial (Coursera, Tutorial)
        https://www.coursera.org
      - Statistics Intermediate Project (Coursera, Project)
        https://www.coursera.org

  Step 3: Improve SQL Querying for Data Extraction and Reporting
  [MET]  Skill: Sql

  Step 4: Improve Python for Data Cleaning and Analysis
  [MET]  Skill: Python

  Step 5: Apply Data Cleaning, Analysis, and Visualisation in a Mini
  [FINAL PROJECT]  Skill: Capstone Project

==================================================================
```

```
# Visual roadmap — uses mpatches for all shapes (FancyBboxPatch and
n   = len(roadmap_steps)
fig, ax = plt.subplots(figsize=(12, n*0.78+1))
ax.set_xlim(0, 10)
ax.set_ylim(0, n+0.5)
ax.axis('off')
ax.set_title(f'Learning Roadmap — {selected_role}',
             fontsize=14, fontweight='bold', pad=15)

for i, s in enumerate(roadmap_steps):
    y     = n - i
    color = STATUS_COLOR.get(s['status'],'#F8FAFC')
    circ  = STATUS_CIRCLE.get(s['status'],'#64748B')

    ax.add_patch(mpatches.FancyBboxPatch(
        (0.2,y-0.38), 9.5, 0.72, boxstyle='round,pad=0.04',
        facecolor=color, edgecolor='#CBD5E1', linewidth=0.8, zorder
    ax.add_patch(mpatches.Circle((0.65,y), 0.26, color=circ, zorder
    ax.text(0.65, y, str(s['step_num']),
            ha='center', va='center', fontsize=8, color='white', fc
    ax.text(1.1, y+0.09, s['action'],
            fontsize=8.5, va='center', color='#1E293B', zorder=2)
```

```
        sub = f"{s['status']}  |  {s['skill']}"
        if s['gap']>0: sub += f"  |  Priority: {s['priority']}"
        ax.text(1.1, y-0.19, sub, fontsize=7.5, va='center', color='#64

    ax.legend(handles=[
        mpatches.Patch(color='#DC2626', label='Major Gap'),
        mpatches.Patch(color='#EA580C', label='Moderate Gap'),
        mpatches.Patch(color='#16A34A', label='Already Met'),
        mpatches.Patch(color='#7C3AED', label='Final Project')
    ], loc='lower right', fontsize=8, framealpha=0.85)
    plt.tight_layout()
    plt.show()
```



Section 13: Full Pipeline — End-to-End Demonstration

This section runs the complete pipeline on a fixed example profile in one readable, structured output. It demonstrates how all five datasets and all eight pipeline stages work together as a cohesive, personalised career guidance system.

Two demonstrations are shown:

1. **First recommended role** — the top ML prediction
2. **Second recommended role** — showing the user can explore alternatives from the top 3

```
def run_full_pipeline(profile_dict, chosen_role=None):
```

```
    """Runs the complete IT Career Planner pipeline from profile to
    print('='*65)
    print('     PERSONALISED IT CAREER PLANNER — FULL PIPELINE')
    print('='*65)

    # Step 1: ML role prediction
    print('\nSTEP 1 — TOP 3 ROLE RECOMMENDATIONS')
    print('-'*45)
    t3, _ = predict_top3_roles(profile_dict)
    print(t3.to_string(index=False))

    # Step 2: Readiness scores
    print('\nSTEP 2 — READINESS SCORES (ALL ROLES)')
    print('-'*45)
    rd = compute_readiness_scores(profile_dict)
    print(rd.to_string(index=False))

    # Step 3: Role selection
    selected = chosen_role if chosen_role else t3['Role'].iloc[0]
    print(f'\nSTEP 3 — SELECTED ROLE : {selected}')

    # Step 4: Role metadata summary
    print('\nSTEP 4 — ROLE SUMMARY')
    print('-'*45)
    meta = role_metadata[role_metadata['role']==selected].iloc[0]
    print(f"  Summary      : {meta['role_summary']}")
    print(f"  Work Type    : {meta['typical_work_type']}")
    print(f"  Career Goal  : {meta['related_career_goal']}")
    print(f"  Key Output   : {meta['key_output']}")

    # Step 5: Why-it-fits explanation
    print('\nSTEP 5 — WHY THIS ROLE FITS')
    print('-'*45)
    print(f'  {why_this_role_fits(profile_dict, selected)}')

    # Step 6: Skill-gap analysis
    print('\nSTEP 6 — SKILL-GAP ANALYSIS')
    print('-'*45)
    gd = compute_skill_gap(profile_dict, selected)
    for _, r in gd[gd['Gap']>0].iterrows():
        print(f"  [GAP] {r['Skill']:28s}  Current:{r['Current Level
              f"Required:{r['Required Level']}  Gap:{r['Gap']}  "
              f"Status:{r['Gap Status']}  Priority:{r['Priority']}"
    for _, r in gd[gd['Gap']==0].iterrows():
        print(f"  [MET] {r['Skill']}")

    # Step 7: Roadmap and resources combined
    print('\nSTEP 7 — PERSONALISED ROADMAP AND RESOURCES')
```

```
    print('-'*45)
    steps = generate_roadmap_with_resources(gd, selected)
    for s in steps:
        badge = f"[{s['status'].upper()}]"
        print(f"\n  Step {s['step_num']}: {s['action']}  {badge}")
        if s['status'] not in ['No Gap','Final Project'] and not s[
            for _, res in s['resources'].iterrows():
                print(f"    - {res['resource_title']} | {res['platf
                print(f"      {res['url']}")

    print('\n'+'='*65)
    print('Pipeline complete.')
    print('='*65)

# Demo 1: Top predicted role
run_full_pipeline(example_profile)
```

```
            Web Developer           76.2
        Software Engineer           76.0
Machine Learning Engineer           65.7
    Cloud/DevOps Engineer           61.5

STEP 3 — SELECTED ROLE : Data Analyst

STEP 4 — ROLE SUMMARY
---------------------------------------------
  Summary      : Collects, cleans, analyses, and visualises data t
  Work Type    : Analytical
  Career Goal  : Work with Data
  Key Output   : reports; dashboards; business insights

STEP 5 — WHY THIS ROLE FITS
---------------------------------------------
  This role fits your profile because you show strong interest in 

STEP 6 — SKILL-GAP ANALYSIS
---------------------------------------------
  [GAP] Data Visualisation            Current:4  Required:5  Gap:1
  [GAP] Statistics                    Current:3  Required:4  Gap:1
  [MET] Sql
  [MET] Python
  [MET] Programming Fundamentals
  [MET] Git
  [MET] Machine Learning
  [MET] Web Development
  [MET] Cloud
  [MET] Devops

STEP 7 — PERSONALISED ROADMAP AND RESOURCES
---------------------------------------------

  Step 1: Develop Dashboard and Visualisation Skills for Insight C
```

```
      - Data Visualisation Advanced Learning Path | Microsoft Learn
        https://learn.microsoft.com/en-us/training/
      - Data Visualisation Advanced Tutorial | YouTube | Tutorial
        https://www.youtube.com/
      - Data Visualisation Advanced Project | Coursera | Project
        https://www.coursera.org

    Step 2: Strengthen Statistics for Data Interpretation  [MODERATE
      - Statistics Intermediate Learning Path | Coursera | Learning |
        https://www.coursera.org
      - Statistics Intermediate Tutorial | Coursera | Tutorial
        https://www.coursera.org
      - Statistics Intermediate Project | Coursera | Project
        https://www.coursera.org

    Step 3: Improve SQL Querying for Data Extraction and Reporting

    Step 4: Improve Python for Data Cleaning and Analysis  [NO GAP]

    Step 5: Apply Data Cleaning, Analysis, and Visualisation in a Mi

  ================================================================
  Pipeline complete.
  ----------------------------------------------------------------
```

```
# Demo 2: Second recommended role — shows the learner can explore a
t3, _ = predict_top3_roles(example_profile)
print(f'Demonstrating pipeline for 2nd recommended role: {t3["Role"
run_full_pipeline(example_profile, chosen_role=t3['Role'].iloc[1])
```

```
  STEP 4 — ROLE SUMMARY
  ----------------------------------------------
    Summary      : Uses data, statistics, and machine learning to bu
    Work Type    : Research-Oriented
    Career Goal  : Build AI Systems
    Key Output   : predictive models; experiments; analytical insigh

  STEP 5 — WHY THIS ROLE FITS
  ----------------------------------------------
    This role fits your profile because you show strong interest in

  STEP 6 — SKILL-GAP ANALYSIS
  ----------------------------------------------
    [GAP] Machine Learning              Current:2  Required:5  Gap:3
    [GAP] Statistics                    Current:3  Required:5  Gap:2
    [GAP] Python                        Current:4  Required:5  Gap:1
    [GAP] Programming Fundamentals      Current:3  Required:4  Gap:1
    [GAP] Cloud                         Current:1  Required:2  Gap:1
    [MET] Sql
    [MET] Data Visualisation
    [MET] Git
    [MET] Web Development
```

```
  [MET] Devops

STEP 7 — PERSONALISED ROADMAP AND RESOURCES
-------------------------------------------

  Step 1: Develop Machine Learning Skills for Classification and P
    - Machine Learning Beginner Learning Path | Kaggle Learn | Lea
      https://www.kaggle.com/learn
    - Machine Learning Beginner Tutorial | Coursera | Tutorial
      https://www.coursera.org
    - Machine Learning Beginner Project | Coursera | Project
      https://www.coursera.org

  Step 2: Improve Statistics for Predictive Analysis and Experimen
    - Statistics Intermediate Learning Path | Coursera | Learning
      https://www.coursera.org
    - Statistics Intermediate Tutorial | Coursera | Tutorial
      https://www.coursera.org
    - Statistics Intermediate Project | Coursera | Project
      https://www.coursera.org

  Step 3: Strengthen Python for Data Processing and Model Developm
    - Python Advanced Learning Path | Kaggle Learn | Learning Path
      https://www.kaggle.com/learn
    - Python Advanced Tutorial | Coursera | Tutorial
      https://www.coursera.org
    - Python Advanced Project | Coursera | Project
      https://www.coursera.org

  Step 4: Improve SQL for Data Access and Feature Preparation  [NO

  Step 5: Apply Data Preparation and Modelling in a Small Data Sci

==================================================================
Pipeline complete.
==================================================================
```

Section 14: Interactive Colab Widget UI

The interactive widget interface allows a learner to set their own profile using sliders and dropdowns, then receive personalised recommendations in real time.

How to use:

1. Adjust the skill and interest sliders to reflect your current level (1 = Beginner, 5 = Expert)
2. Select your preferred work type, learning style, and career goal
3. Click **Run Planner** to see your top 3 recommended roles and readiness scores
4. Select any of the 3 recommended roles from the dropdown
5. Click **Explore Role** to see the full skill-gap analysis, why-it-fits explanation, roadmap, and resources

```
display(HTML("""
<div style="background:linear-gradient(135deg,#1E40AF,#6D28D9);
            padding:20px 24px;border-radius:12px;margin-bottom:16px;
            box-shadow:0 4px 20px rgba(30,64,175,0.3);">
  <h2 style="color:white;margin:0;font-family:Arial;font-size:22px;f
    🎯 Personalised IT Career Planner
  </h2>
  <p style="color:#C7D2FE;margin:8px 0 0;font-family:Arial;font-size
    Set your skill levels, interest areas, and preferences below.<br
    Click <b>▶ Run Planner</b> to see your top 3 role recommendation
    then select a role and click <b>🔍 Explore Role</b> for your ful
  </p>
</div>
"""))

S  = {'description_width':'230px'}
LW = widgets.Layout(width='490px')

# Skill sliders
s_py  = widgets.IntSlider(value=3,min=1,max=5,description='🐍 Python
s_sql = widgets.IntSlider(value=3,min=1,max=5,description='🗄️  SQL',
s_st  = widgets.IntSlider(value=3,min=1,max=5,description='📐 Statis
s_ml  = widgets.IntSlider(value=2,min=1,max=5,description='🤖 Machin
s_pg  = widgets.IntSlider(value=3,min=1,max=5,description='💻 Progra
s_wd  = widgets.IntSlider(value=2,min=1,max=5,description='🌐 Web De
s_cl  = widgets.IntSlider(value=1,min=1,max=5,description='☁️  Cloud
s_dv  = widgets.IntSlider(value=1,min=1,max=5,description='⚙️  DevOp
```

```
s_dz  = widgets.IntSlider(value=3,min=1,max=5,description='📊 Data V
s_gt  = widgets.IntSlider(value=2,min=1,max=5,description='🔀 Git',

# Interest sliders
i_da  = widgets.IntSlider(value=4,min=1,max=5,description='📊 Intere
i_ai  = widgets.IntSlider(value=3,min=1,max=5,description='🤖 Intere
i_sd  = widgets.IntSlider(value=2,min=1,max=5,description='💻 Intere
i_ww  = widgets.IntSlider(value=2,min=1,max=5,description='🌐 Intere
i_ci  = widgets.IntSlider(value=1,min=1,max=5,description='☁️  Inter
i_bp  = widgets.IntSlider(value=4,min=1,max=5,description='💼 Intere

# Preference dropdowns
dd_wt = widgets.Dropdown(
    options=['Analytical','Research-Oriented','System Building',
             'User-Facing Development','Automation-Focused','Infrast
    value='Analytical', description='⚙️  Preferred Work Type:', styl
dd_ls = widgets.Dropdown(
    options=['Visual','Practical','Project-Based','Theory-First'],
    value='Visual', description='📖 Learning Style:', style=S, layou
dd_cg = widgets.Dropdown(
    options=['Exploring Options','Work with Data','Build AI Systems'
             'Build Software','Build Websites','Manage Cloud/Deploym
    value='Work with Data', description='🎯 Career Goal:', style=S,

# Role selector — populated after Run Planner
dd_role = widgets.Dropdown(
    options=['Run Planner first'], value='Run Planner first',
    description='📌 Select Role:', style=S, layout=LW, disabled=True

# Action buttons
btn_run    = widgets.Button(description='▶  Run Planner', button_sty
                            layout=widgets.Layout(width='170px',hei
btn_detail = widgets.Button(description='🔍 Explore Role', button_st
                            layout=widgets.Layout(width='170px',hei
out_top    = widgets.Output()
out_detail = widgets.Output()

def build_profile():
    """Builds a profile dict from current widget values."""
    return {
        'interest_data_analysis':i_da.value, 'interest_ai_ml':i_ai.v
        'interest_software_development':i_sd.value, 'interest_web_de
        'interest_cloud_infrastructure':i_ci.value, 'interest_busine
        'skill_python':s_py.value, 'skill_sql':s_sql.value, 'skill_s
        'skill_machine_learning':s_ml.value, 'skill_programming_fund
        'skill_web_development':s_wd.value, 'skill_cloud':s_cl.value
        'skill_data_visualisation':s_dz.value, 'skill_git':s_gt.valu
        'preferred_work_type':dd_wt.value, 'preferred_learning_style
```

```
            'career_goal':dd_cg.value
        }

    def on_run(b):
        p = build_profile(); btn_run._profile = p
        with out_top:
            clear_output(wait=True)
            t3, _ = predict_top3_roles(p)
            rd     = compute_readiness_scores(p)
            display(HTML('<h3 style="color:#1E40AF;margin-bottom:6px;">🏆
            display(t3)
            display(HTML('<h3 style="color:#1E40AF;margin:14px 0 6px;">📊
            display(rd)
            roles = t3['Role'].tolist()
            dd_role.options=roles; dd_role.value=roles[0]
            dd_role.disabled=False; btn_detail.disabled=False
        with out_detail: clear_output()

    def on_explore(b):
        p = btn_run._profile; sel = dd_role.value
        with out_detail:
            clear_output(wait=True)
            display(HTML(f'<h3 style="color:#6D28D9;">🎯 Exploring: {sel

            # Role metadata card
            meta = role_metadata[role_metadata['role']==sel].iloc[0]
            display(HTML(f"""
            <div style="background:#F5F3FF;border-left:4px solid #7C3AED
                        padding:12px 16px;border-radius:8px;margin:8px 0
              <b>📝 Role Summary:</b> {meta['role_summary']}<br><br>
              <b>Work Type:</b> {meta['typical_work_type']}  |
              <b>Career Goal:</b> {meta['related_career_goal']}<br>
              <b>Key Output:</b> {meta['key_output']}
            </div>"""))

            # Why-it-fits explanation card
            explanation = why_this_role_fits(p, sel)
            display(HTML(f"""
            <div style="background:#EFF6FF;border-left:4px solid #2563EB
                        padding:12px 16px;border-radius:8px;margin:8px 0
              <b>💡 Why This Role Fits:</b><br>
              <span style="color:#1E293B;">{explanation}</span>
            </div>"""))

            # Skill-gap table
            gd = compute_skill_gap(p, sel)
            display(HTML('<h4 style="color:#DC2626;margin:12px 0 6px;">
            display(gd)
```

```
# Skill-gap chart
fig, ax = plt.subplots(figsize=(10,4))
x = np.arange(len(gd)); w=0.36
ax.bar(x-w/2,gd['Current Level'], w,color=GAP_CURRENT,edgeco
ax.bar(x+w/2,gd['Required Level'],w,color=GAP_REQUIRED,edgec
for i,row in gd.iterrows():
    if row['Gap']>0:
        ax.annotate(f"Gap:{int(row['Gap'])}",
                    xy=(i,max(row['Current Level'],row['Requ
                    ha='center',fontsize=8,color='#DC2626',f
ax.set_xticks(x); ax.set_xticklabels(gd['Skill'],rotation=30
ax.set_title(f'Skill-Gap — {sel}',fontsize=11,fontweight='bo
ax.set_ylim(0,7); ax.legend(fontsize=9)
ax.yaxis.grid(True,linestyle='--',alpha=0.4); ax.set_axisbel
plt.tight_layout(); plt.show()

# Gap summary cards
strengths = gd[gd['Gap']==0]['Skill'].tolist()
moderate  = gd[gd['Gap Status']=='Moderate Gap']['Skill'].to
major     = gd[gd['Gap Status']=='Major Gap']['Skill'].tolis
display(HTML(f"""
<div style="display:flex;gap:12px;margin:12px 0;">
  <div style="flex:1;background:#DCFCE7;border-radius:8px;pa
              border-left:4px solid #16A34A;">
    <b style="color:#16A34A;">✅ Strengths</b><br>
    <span style="font-size:12px;color:#1E293B;">{', '.join(s
  </div>
  <div style="flex:1;background:#FEF9C3;border-radius:8px;pa
              border-left:4px solid #EA580C;">
    <b style="color:#EA580C;">⚠️ Moderate Gaps</b><br>
    <span style="font-size:12px;color:#1E293B;">{', '.join(m
  </div>
  <div style="flex:1;background:#FEE2E2;border-radius:8px;pa
              border-left:4px solid #DC2626;">
    <b style="color:#DC2626;">❌ Major Gaps</b><br>
    <span style="font-size:12px;color:#1E293B;">{', '.join(m
  </div>
</div>"""))

# Roadmap and resources
display(HTML('<h4 style="color:#1E40AF;margin:12px 0 6px;">🗺
steps = generate_roadmap_with_resources(gd, sel)
for s in steps:
    bg   = STATUS_COLOR.get(s['status'],'#F8FAFC')
    circ = STATUS_CIRCLE.get(s['status'],'#64748B')
    sub  = f"{s['status']}"
    if s['gap']>0: sub += f" | Priority: {s['priority']}"
    display(HTML(f"""
```

```
                <div style="background:{bg};border-radius:8px;padding:10
                            margin:6px 0;border-left:4px solid {circ};">
                  <b>Step {s['step_num']}: {s['action']}</b>
                  <span style="color:#64748B;font-size:12px;"> [{sub}]</
                </div>"""))
                if s['status'] not in ['No Gap','Final Project'] and not
                    res_html = '<div style="padding:4px 0 8px 20px;">'
                    for _, res in s['resources'].iterrows():
                        res_html += (f'📚 <a href="{res["url"]}" target=
                                     f'style="color:#2563EB;font-weight:
                                     f'{res["resource_title"]}</a> '
                                     f'<span style="color:#64748B;font-s
                                     f'({res["platform"]} | {res["resour
                    res_html += '</div>'
                    display(HTML(res_html))

btn_run.on_click(on_run)
btn_detail.on_click(on_explore)

# Two-column layout
skill_box = widgets.VBox([
    widgets.HTML('<b style="color:#1E40AF;font-size:14px;">🛠️ Skill
    s_py,s_sql,s_st,s_ml,s_pg,s_wd,s_cl,s_dv,s_dz,s_gt
])
interest_box = widgets.VBox([
    widgets.HTML('<b style="color:#1E40AF;font-size:14px;">💡 Intere
    i_da,i_ai,i_sd,i_ww,i_ci,i_bp
])
pref_box = widgets.VBox([
    widgets.HTML('<b style="color:#1E40AF;font-size:14px;">⚙️ Prefer
    dd_wt, dd_ls, dd_cg,
    widgets.HTML('<br>'), btn_run,
    widgets.HTML('<br>'),
    widgets.HTML('<b style="color:#6D28D9;font-size:13px;">📌 Select
    dd_role, btn_detail
])

display(
    widgets.HBox([skill_box, widgets.VBox([interest_box, pref_box])]
    widgets.HTML('<hr style="margin:18px 0;border-color:#E2E8F0;">')
    out_top, out_detail
)
```

🎯 Personalised IT Career Planner

Set your skill levels, interest areas, and preferences below.
Click ▶ **Run Planner** to see your top 3 role recommendations, then select a role and

click 🔍 **Explore Role** for your full personalised roadmap.

🛠️ Skill Levels (1=Beginner, 5=Expert)

🐍 Python

🗄️ SQL

📐 Statistics

🤖 Machine Learning

💻 Programming Fundamentals

🌐 Web Development

☁️ Cloud

⚙️ DevOps

📊 Data Visualisation

🔀 Git

💡 Interest Levels (1=Low, 5=High)

📊 Interest: Data Analysis

🤖 Interest: AI / ML

💻 Interest: Software Dev

🌐 Interest: Web Dev

☁️ Interest: Cloud

💼 Interest: Business Problems

⚙️ Preferences and Actions

⚙️ Preferred Work Type: Analy

📖 Learning Style: Visua

🎯 Career Goal: Work

▶ Run Planner

📌 Select a role to explore:

📌 Select Role: Data

🔍 Explore Role

🏆 Top 3 Recommended Roles

	Recommendation	Role
0	1st Recommendation	Data Analyst
1	2nd Recommendation	Data Scientist
2	3rd Recommendation	Web Developer

📈 Readiness Scores

```
Role  Readiness (%)
```

0	Web Developer	76.2
1	Data Analyst	75.9
2	Software Engineer	72.0
3	Data Scientist	64.7
4	Machine Learning Engineer	60.0
5	Cloud/DevOps Engineer	57.7

🎯 Exploring: Data Analyst

📝 **Role Summary:** Collects, cleans, analyses, and visualises data to support decision-making.

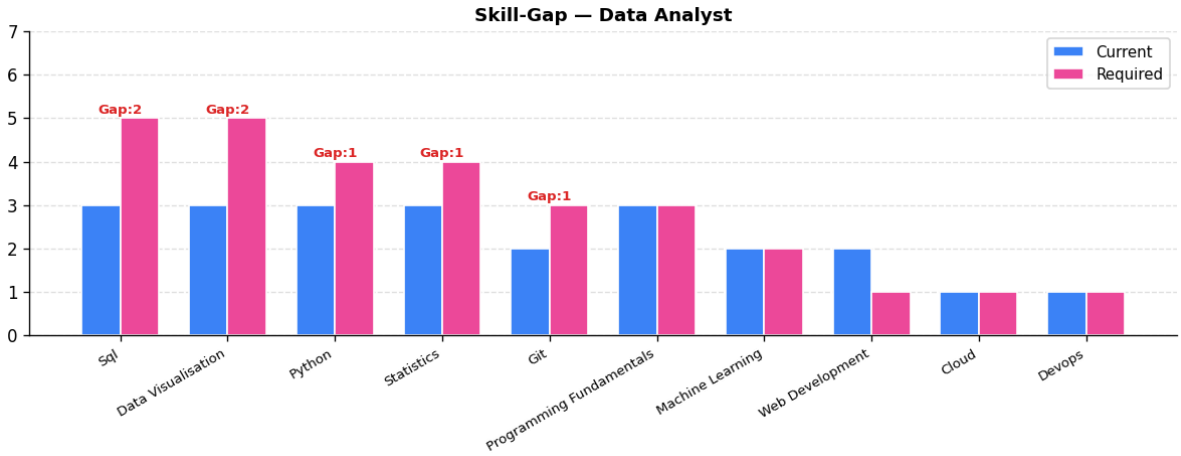
Work Type: Analytical | **Career Goal:** Work with Data
Key Output: reports; dashboards; business insights

💡 **Why This Role Fits:**
This role fits your profile because you show strong interest in data analysis, solid SQL ability, and a preference for analytical work. Your career goal to 'work with data' also aligns directly with what Data Analysts do day-to-day.

🔍 Skill-Gap Analysis

	Skill	Current Level	Required Level	Gap	Gap Status	Priority
0	Sql	3	5	2	Major Gap	High
1	Data Visualisation	3	5	2	Major Gap	High
2	Python	3	4	1	Moderate Gap	High
3	Statistics	3	4	1	Moderate Gap	High
4	Git	2	3	1	Moderate Gap	Medium
5	Programming Fundamentals	3	3	0	No Gap	Low
6	Machine Learning	2	2	0	No Gap	Low
7	Web Development	2	1	0	No Gap	Low
8	Cloud	1	1	0	No Gap	Low

9	Devops	1	1	0	No Gap	Low



✅ **Strengths**
Programming Fundamentals, Machine Learning, Web Development, Cloud, Devops

⚠️ **Moderate Gaps**
Python, Statistics, Git

❌ **Major Gaps**
Sql, Data Visualisation

🗺️ Personalised Roadmap and Resources

Step 1: Improve SQL Querying for Data Extraction and Reporting [Major Gap | Priority: High]

📚 SQL Intermediate Learning Path (Kaggle Learn | Learning Path)
📚 SQL Intermediate Tutorial (Coursera | Tutorial)
📚 SQL Intermediate Project (Coursera | Project)

Step 2: Develop Dashboard and Visualisation Skills for Insight Communication [Major Gap | Priority: High]

📚 Data Visualisation Intermediate Learning Path (Microsoft Learn | Learning Path)
📚 Data Visualisation Intermediate Tutorial (YouTube | Tutorial)
📚 Data Visualisation Intermediate Project (Coursera | Project)

Step 3: Improve Python for Data Cleaning and Analysis [Moderate Gap | Priority: High]

📚 Python Intermediate Learning Path (Kaggle Learn | Learning Path)
📚 Python Intermediate Tutorial (Coursera | Tutorial)
📚 Python Intermediate Project (Coursera | Project)

Section 15: Conclusion, Limitations and Future Work

What Was Achieved

This project successfully delivered an end-to-end Personalised IT Career Planner that addresses the stated problem of unclear IT career pathways for students and self-learners. The system integrates supervised ML classification, rule-based explanation, skill-gap analysis with priority classification, personalised roadmap generation, and curated resource recommendations across five self-constructed, interconnected datasets.

The best performing model (XGBoost) achieved a macro F1-score of 0.9975 on the held-out test set, confirmed by 5-fold stratified cross-validation. All six roles achieved near-perfect per-class F1 scores, demonstrating that the feature design and dataset construction are effective.

Fulfilment of Objectives

Objective	Status	Evidence
Identify and model IT role requirements	Achieved	role_skill_matrix.csv + ML pipeline
Train and compare supervised ML models	Achieved	Decision Tree, Random Forest, XGBoost compared wi
Skill-gap analysis with priority classification	Achieved	Gap status (No/Moderate/Major) + Priority column
Personalised roadmap generation	Achieved	Steps ordered by gap priority, clean numbering
Curated resource recommendations	Achieved	300 resources matched by skill, level, role
Interactive demonstration	Achieved	Full Colab widget UI + deployed Streamlit web applica

Limitations

- **Synthetic dataset:** All datasets are self-constructed. Real-world learner profile data collected via surveys or online platforms would provide stronger external validity and generalisability.
- **Overconfident model probabilities:** The model is trained on synthetic data where the target label is partially deterministic, which can produce overconfident predictions. This was addressed through softmax normalisation, but a real-world deployment would benefit from probability calibration.
- **Rule-based explanation:** The why-it-fits explanation is template-driven and rule-based. A more advanced NLP-based approach (e.g. using a language model) could produce richer, more nuanced explanations.
- **Fixed role taxonomy:** The system covers six fixed IT roles. Real career landscapes include many more specialisations and sub-roles.

- **Static resource catalogue:** The 300-resource catalogue is manually curated and static. A live system would benefit from dynamic resource retrieval via APIs.

Future Work

- Replace the synthetic dataset with real learner profile data collected via structured surveys.
- Integrate real-time job market APIs (e.g. LinkedIn, Indeed) to dynamically update role demand and resource recommendations.
- Implement SHAP-based explainability to provide feature-level justification for each ML prediction.
- Extend the role taxonomy to include sub-specialisations (e.g. NLP Engineer, Frontend Developer, Site Reliability Engineer).
- Add a progress tracking feature so learners can update their profile over time and monitor roadmap completion.

Section 16: GitHub Upload

Upload the notebook and all project files to GitHub for version control and submission.

```
import json, re

# Updated path with correct filename
path = '/content/drive/MyDrive/IT_Career_Planner_Project/25036746_D

with open(path, 'r') as f:
    content = f.read()

# Remove any ghp_ tokens using pattern matching
content = re.sub(r'ghp_[A-Za-z0-9]+', 'YOUR_GITHUB_TOKEN', content)

# Remove widgets metadata that causes Invalid Notebook preview on G
nb = json.loads(content)
if 'widgets' in nb.get('metadata', {}):
    del nb['metadata']['widgets']

with open(path, 'w') as f:
    json.dump(nb, f, indent=1)

print('Tokens removed and notebook fixed.')
```

```
Tokens removed and notebook fixed.
```

```
import shutil, os

!rm -rf /content/it-career-planner
os.makedirs('/content/it-career-planner', exist_ok=True)
DRIVE = '/content/drive/MyDrive/IT_Career_Planner_Project'

# Updated filename to match your actual notebook name
for f in ['25036746_DSAI_Individual_Project.ipynb','user_profiles.c
          'role_skill_matrix.csv','role_metadata.csv',
          'roadmap_templates.csv','resource_catalogue.csv']:
    shutil.copy(f'{DRIVE}/{f}', f'/content/it-career-planner/{f}')
    print(f'Copied: {f}')
print('All files ready.')
```

```
Copied: 25036746_DSAI_Individual_Project.ipynb
Copied: user_profiles.csv
Copied: role_skill_matrix.csv
Copied: role_metadata.csv
Copied: roadmap_templates.csv
Copied: resource_catalogue.csv
All files ready.
```

```
import os
os.chdir('/content/it-career-planner')
!git init
!git config --global user.name "HeinHtet-Phyo"
!git config --global user.email "heinhtetphyo56@gmail.com"
!git remote add origin https://YOUR_GITHUB_TOKEN@github.com/HeinHte
!git add .
!git commit -m "Update IT Career Planner notebook"
!git branch -M main
!git push -u origin main --force
print('Uploaded to GitHub!')
```

```
hint: Using 'master' as the name for the initial branch. This defaul
hint: is subject to change. To configure the initial branch name to
hint: of your new repositories, which will suppress this warning, ca
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk'
hint: 'development'. The just-created branch can be renamed via this
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /content/it-career-planner/.git/
[master (root-commit) e45eb06] Update IT Career Planner notebook
 6 files changed, 9835 insertions(+)
 create mode 100644 25036746_DSAI_Individual_Project.ipynb
 create mode 100644 resource_catalogue.csv
 create mode 100644 roadmap_templates.csv
 create mode 100644 role_metadata.csv
 create mode 100644 role_skill_matrix.csv
 create mode 100644 user_profiles.csv
fatal: could not read Password for 'https://YOUR_GITHUB_TOKEN@github
Uploaded to GitHub!
```